



**CPU BUISNESS AND INFORMATION
TECHNOLOGY COLLEGE
DEPARTMENT OF COMPUTER SCIENCE**

Project title: Home Tutor Finder

BY:

Student name:

Id:

Fitsum Tibebe Rcs/524/14

Musab Abduljebar TRcs/1025/14

Ruth Tesfaye Rcs/635/14

Yared Gedion Rcs/414/14

Project Advisor: Ms.'s Axumawit Tasew

Feb, 2025

DECLARATION

This is to declare that this project work which is done under the supervision of Axumawit Tasew and having the title Home Tutor finder is the sole contribution of:

Musab Abduljebar

Fitsum Tibebe

Yared Gediya

Ruth Tesfaye

No part of the project work has been reproduced illegally (copy and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if violation of this declaration is proven.

Date: _____

Group Members:

Full Name

Signature

CERTIFICATE

I certify that this BSc final project report entitled title Home Tutor finder by:

Musab Abduljebar

Fitsum Tibebe

Yared Gediya

Ruth Tesfaye

is approved by me for submission. I certify further that, to the best of my knowledge, the report represents work carried out by the students.

Date Name and Signature of Supervisor

ACKNOWLEDGEMENT

We are most grateful to the Almighty God for His infinite mercy and love that guided us throughout the duration of the program. We deeply grateful to our advisor Me A, for his precious and constructive comments and suggestions during the course of this project. In addition to his contribution to this project, we also extremely indebted to our family for their support and encouragement throughout our life. We are also grateful to our friends for their moral support since the beginning of our education. Last but not least, we would like to express our deep gratitude to staff of Africa insurance share company for their willingness to cooperate for a data collection.

Table of Contents

DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGEMENT	iv
List of figures	viii
List of tables	ix
CHAPTER ONE	1
INTRODUCTION	1
1.1 Background Information	1
1.2 Statement of the Problem	1
1.3.1 General Objectives	1
1.3.2 Specific Objectives	1
1.4 Scope of the Project	2
1.5.2 System Development Methodology	3
1.5.3 Development Tools	3
1.6 Beneficiaries	3
1.7 Schedule	4
CHAPTER TWO	6
REQUIREMENT ANALYSIS	6
2.1 Introduction	6
2.2 Current System Overview	6
2.3 Proposed System Overview	6
2.3.1 Functional Requirements	6
2.3.2 Non-functional Requirements	8
2.4 System Models - Requirement Determination	8
2.4.1 Essential Use Case Modeling	8
2.4.1.1 Use Case Diagram	8
2.4.1.2 Use Case Documentation	10
2.4.2 Essential UI Prototype	22
2.4.3 User Interface Flow Diagram	38

2.4.4 Supplementary Specifications.....	39
2.4.4.1 Business Rules	39
2.4.4.2 Constraints	42
2.4.4.3 Change Case.....	42
2.5 System Models - Analysis	42
2.5.1 System Use Case Modeling	42
2.5.1.1 Use Case Diagram.....	42
2.5.1.2 Use Case Documentation.....	44
2.5.2 Sequence Diagram	55
2.5.3 Activity Diagram	69
CHAPTER THREE	74
SYSTEM DESIGN	74
3.1 Introduction.....	74
3.2 Proposed software Architecture.....	74
3.2.1 Overview.....	74
3.2.2 System Decomposition	75
3.2.3 Hardware and Software Mapping.....	78
3.2.4 Persistent Data management.....	79
3.2.5 Access Control and security.....	79
3.2.6 Subsystem Services.....	80
3.2.7 Detail Class Diagram	81
3.2.8 Package	82
CHAPTER FOUR.....	84
IMPLEMENTATION.....	84
4.1 Introduction.....	84
4.2 Algorithms	84
4.2.1 Tutor Search Algorithm	84
Steps of the algorithm	84
Pseudocode	Error! Bookmark not defined.
4.2.2 Session Scheduling Algorithm.....	84
Steps.....	84

Pseudocode	Error! Bookmark not defined.
4.3 Description of Coding Tools.....	85
4.3.1 Programming Language.....	85
4.3.2 Django Framework	85
4.3.3 Frontend Technologies.....	85
4.3.4 Database System	85
4.3.5 Version Control.....	86
4.3.6 Development Environment	86
4.4 Code for Major Functionalities	86
4.4.1 User Registration and Authentication.....	86
4.4.2 Profile Management.....	87
4.4.3 Availability Management.....	88
4.4.4 Virtual Classroom	89
4.4.7 Payment Integration	90
4.5 Testing.....	90
4.5.1 Unit Testing	90
4.5.2 Integration Testing.....	91
4.5.3 System Testing.....	92
CHAPTER FIVE	93
CONCLUSION AND RECOMMENDATIONS	93
5.1 Introduction.....	93
5.1 Recommendations.....	93
▪ AI-Based Tutor Recommendation	93
▪ Mobile Application Development.....	93
▪ Improved Security Features	93
▪ AI-Assisted Learning Tools.....	93
5.2 Conclusion	94

List of figures

Figure I Gant-chart.....	5
Figure II Use case Diagram	9
Figure III UI prototype.....	22
Figure IV UI prototype	23
Figure V user interface flow diagram	38
Figure VI Use Case diagram.....	43
Figure VIII Tutor Accept Payment Sequence Diagram.....	55
Figure IX Student Make Payment Sequence Diagram	55
Figure X Student Search For Tutors Sequence Diagram.....	56
Figure XI Student Rate and Review Tutor Sequence Diagram	57
Figure XII Student Access Materials Sequence Diagram.....	58
Figure XIII Tutor Upload Materials Sequence Diagram	59
Figure XIV Tutor Track Earnings Sequence Diagram	59
Figure XV Lesson Reset Sequence Diagram.....	60
Figure XVI Logout Sequence Diagram	61
Figure XVII Tutor Set Availability Sequence Diagram	62
Figure XVIII Admin Ban User Sequence Diagram.....	63
Figure XIX Admin Approve Tutor Registration	64
Figure XX Admin Resolve Disputes Sequence Diagram	65
Figure XXI Profile Update Sequence Diagram	66
Figure XXII Login Sequence Diagram.....	67
Figure XXIII Reset Password Sequence Diagram.....	68
Figure XXIV Session Type Switch Sequence Diagram	69
Figure XXV student Activity diagram.....	70
Figure XXVI Admin Activity diagram.....	71
Figure XXVII Tutor activity diagram.....	73
figure XXVIII system architecture	75
Figure XXIX component diagram	76
Figure XXX Hardware to software mapping.....	78
Figure XXXI class diagram	82
Figure XXXII package diagram.....	83

List of tables

Table 1 Essential use cases	10
Table 2 User interfaces	24
Table 3 business Rules.....	39
Table 4System use cases	44
Table 5Access control.....	80

CHAPTER ONE

INTRODUCTION

1.1 Background Information

In today's fast-paced world, finding reliable and skilled home tutors has become increasingly challenging for students and parents. Traditional methods of searching for tutors—through word of mouth, ads, or local agencies can be inefficient, often leaving students with limited options or unreliable sources. With the rise of digital technology and the growing demand for personalized education, there is an opportunity to create a platform that streamlines the process of finding and engaging home tutors.

1.2 Statement of the Problem

Finding the right tutor with the required skills, availability, and affordability is often a time-consuming and stressful process for both students and parents. Moreover, there is a lack of platforms that allow for comprehensive tutor-student interaction, including virtual and in-person teaching, material sharing, and group organization. Existing solutions do not fully address the need for a transparent, reliable, and easily accessible system where students can evaluate tutors based on ratings, view detailed profiles, and ensure fair commission payments. Currently some local and international systems available for solving this problem e.g (tutor Ethiopia and learn pick), but they lose some or more features of more students and tutors desired system.

1.3 Objectives

1.3.1 General Objectives

The general objective of this project is to design and develop a comprehensive home tutor finding platform that connects students with qualified tutors, facilitates transparent and reliable transactions, and enhances their academic experience and advancing their learning outcomes

1.3.2 Specific Objectives

- User Registration and profiles: allow students and tutors to create accounts with their detailed profiles, including academic background, and subject of expertise.
- Search and Match: enable students to find tutors based on subject, expertise, availability. Offer a matching algorithm to suggest suitable tutors for students

- Tutor Listing: Display comprehensive tutor listing with profiles showcasing qualifications, experience, teaching style, availability, and user ratings,
- Messaging and communication: provide an in-app messaging system for students and tutors to communicate, discuss academic needs, schedule sessions and exchange important information.
- Virtual Class: integrate video conferencing tools for online tutoring sessions.
- Payment Gateway: integrate a secure payment gateway for students to pay for tutoring sessions within the platform. Provide transparency on pricing and billing information.
- Review and rating: incorporate a review and rating system for students to provide feedback on tutoring sessions, helping other users make informed decisions when selecting tutors.
- Resource sharing: allow tutors to share educational resources, study materials, practice exams etc. to enhance the learning experience outside tutoring sessions
- Profile management: enable users to update their profiles, manage availability, set preference and track session history for better organization and engagement.

1.4 Scope of the Project

The scope of this project includes the design, development, and deployment of a web-based application that supports the functionalities mentioned above. The system will cater to both students and tutors, enabling a seamless interaction for booking, reviewing, and attending lessons, either physically or virtually. The app will operate on a commission-based model, and it will be available for use by students and tutors globally, though initially, it may target specific regions.

The project will not cover the development of any hardware components but will rely on smartphones, tablets, and computers for user access.

1.5 Tools and Methodologies

1.5.1 Data Collection Methodologies

Data collection for this project will involve:

- Surveys and interviews: Gathering feedback from potential users (students and tutors) to understand their needs, preferences, and pain points.
- Market research: Analyzing competitor platforms to identify gaps and unique features that can give our app a competitive edge.
- User testing: Conducting usability testing to refine the app's features and ensure a user-friendly interface.

1.5.2 System Development Methodology

The development will follow the Iterative Waterfall methodology. It's similar to the regular Waterfall model but gives us more flexibility to make improvements as we go. The process will still follow clear steps starting with requirement analysis, then design, implementation, testing, and finally deployment but after each phase, we'll review what we've done and make changes if needed before moving forward.

This approach helps us spot and fix issues early, instead of waiting until the very end. We'll start by developing the core features like user profiles, search functionality, and payment systems. Once those parts are tested and working well, we'll move on to adding more advanced features such as live streaming, group management, and material sharing.

1.5.3 Development Tools

- The front end is the interface part, which is visible for the end users. For this part the tool we planned to use is typical HTML5, CSS and HTMX for dynamic behavior. We choose this because this tool is a very simple, and easy to develop user-friendly graphical interface.
- The backend Development and the database part, which is invisible for end users. Planned to use a python frame work called Django, and SQLite database storage respectively. We choose Django because it has built in functionalities like security, session management, user defined functions, and stored procedures that speedup the development phase.
- For payment and online streaming purpose we considered to use Third-party APIs, Chapa for payment and Jitsi for video conferencing.
- For dynamic scheduling and availability settings we planned to use Full calendar, a JS library for calendar integration.
- Telegram integration: Since the platform is a web App, the user should get notification on any updates without the need to login the website , and get any notification on their phone(telegram app) instantly.

1.6 Beneficiaries

The primary beneficiaries of this project will be:

- Students, who will be benefited from having an easy-to-use platform to find and connect with tutors based on specific needs such as subject expertise, availability, and price.
- Tutors, who will gain access to a large pool of potential students, be able to create and manage their profiles, and conduct lessons either in-person or online.
- Educational Institutions: That could partner with the app to provide supplementary learning materials or additional tutoring resources.
- The Development Team: Who will gain experience in creating a market-driven application with a real-world user base.

1.7 Schedule

The project will be developed over the following phases:

- Phase 1: Requirements Gathering and Analysis (1 month): Collect data from potential users and stakeholders, define system requirements.
- Phase 2: Design and Architecture (2 months): Develop wireframes, prototypes, and system architecture.
- Phase 3: Iterative Development (4 months):
 - Iteration 1(2 months): Build and test core features such as tutor and student profiles, search functionality and the payment system.
 - Iteration2 (2 months): Improve based on feedback, then develop advanced features like live streaming, group management, and material sharing. Each iteration includes its own design, implementation and testing cycle.
- Phase 4: Testing and Quality Assurance (1 month): Conduct complete system testing (performance, user acceptance, and security).
- Phase 5: Launch and Deployment (1 month): Final deployment of the app on platforms (Android, Web) and make it available to users.
- Phase 6: Maintenance and Updates (Ongoing): Post-launch monitoring, bug fixes, and introduce new improvements based on user feedback.

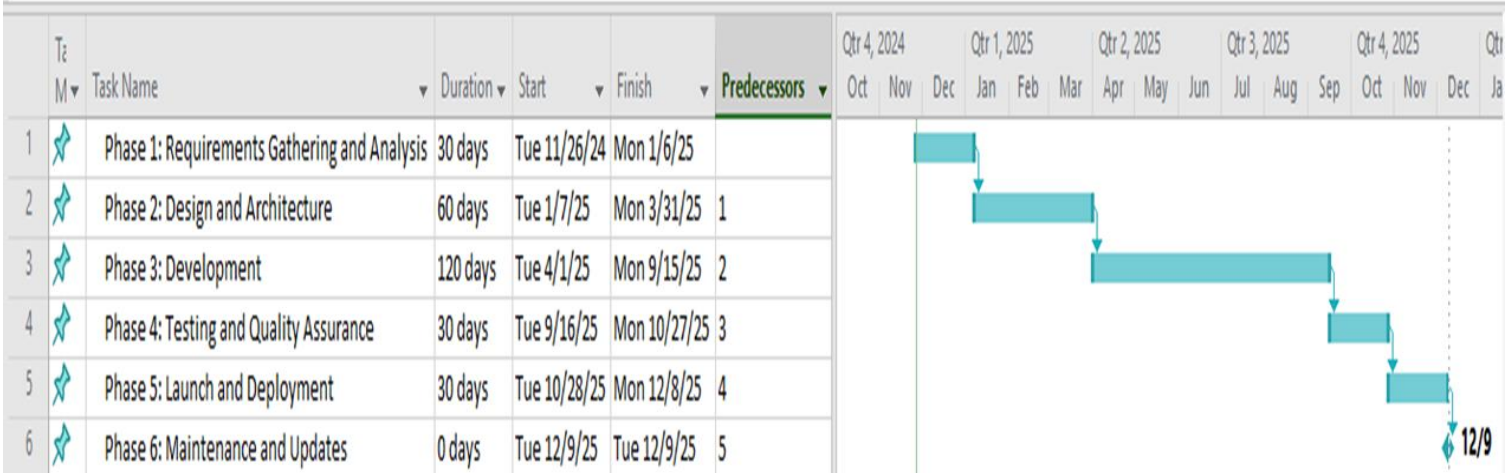


Figure I Gant-chart

CHAPTER TWO

REQUIREMENT ANALYSIS

2.1 Introduction

This chapter discusses the requirements and analysis necessary for developing the Home Tutor-Finding Platform. It evaluates the current system, identifies functional and non-functional requirements, and proposes the system's design. This ensures the platform meets users' needs effectively.

2.2 Current System Overview

The current method for finding home tutors often relies on manual or fragmented processes, such as referrals, social media, or offline advertisements. These systems have limitations:

- Lack of centralized platforms.
- Time-consuming and inefficient tutor matching.
- No system for tracking tutors' qualifications, schedules, or feedback.

Challenges include:

- Difficulty finding qualified tutors for specific subjects or schedules.
- Limited trust due to insufficient tutor verification.
- No system for performance reviews or tutor ratings.
- Currently some local and international systems available for solving this problem e.g (tutor Ethiopia and learn pick), but they lose some or more features of more students and tutors desired system.

2.3 Proposed System Overview

The proposed system aims to address the inefficiencies of the current system by creating a centralized, user-friendly platform for connecting tutors and students.

2.3.1 Functional Requirements

1. Student Features:

- Register/login to the platform.
- Search for tutors based on filters (subject, schedule, etc.).
- View tutor profiles and ratings.

- View tutor availability
- Select tutoring packages
- Attend virtual classes (Jitsi)
- Download learning resources
- Rate and give feedback
- Get Telegram + in-app notifications
- Book sessions and track schedules.

2. Tutor Features:

- Register/login and create a profile.
- Set availability with weekly repeating
- Create and manage packages
- Upload and share resources
- Receive and manage booking requests.
- View and respond to student feedback.

3. Admin Features:

- Manage tutor and student accounts.
- Upload and share resources
- Manage sessions (reschedule/cancel)
- Oversee tutor packages/resources
- Session management tools
- Handle complaints/flags
- Verify tutor profiles and qualifications.
- Monitor platform activity and resolve disputes.

4. System features:

- System should Collect commission from clients.
- System should Prepare virtual platform for online tutoring.
- System should allow material sharing.
- System should perform Matching and searching performance.
- System should register Session information for every user.

- System should Allow users to select role (tutor or student).
- System should support login with client's user name and password.
- System should provide appropriate service based on role and privilege.
- System should send notifications in both the website and through telegram
- System should deny service if clients try to login with wrong credentials.

2.3.2 Non-functional Requirements

- Security and Privacy:,
 - ✓ Encrypted backend data
 - ✓ Only authenticated admin can access the system backend server.
 - ✓ Login to the system is possible only if have unique id.
- User Interface Design: Create an intuitive and user-friendly interface that is easy to navigate and understand, with clear instructions and labels.
- Technical Documentation: Document the system architecture, design decisions, APIs, and data schemas to facilitate maintenance, troubleshooting, and future development.
- Hardware consideration: our web based tutor finding platform can run on smartphone, desktop and tablets have the capability of internet access.
- Performance characteristics: if there is no internet connection error the system must respond for all legal users.
- Error handling and extreme condition: use try and catch mechanism to avoid system crash.
- Physical environment: our web application deployed on server and clients can access through internet.
- System modification: our proposed system can be easily modified or update because all documentation is clear.

2.4 System Models - Requirement Determination

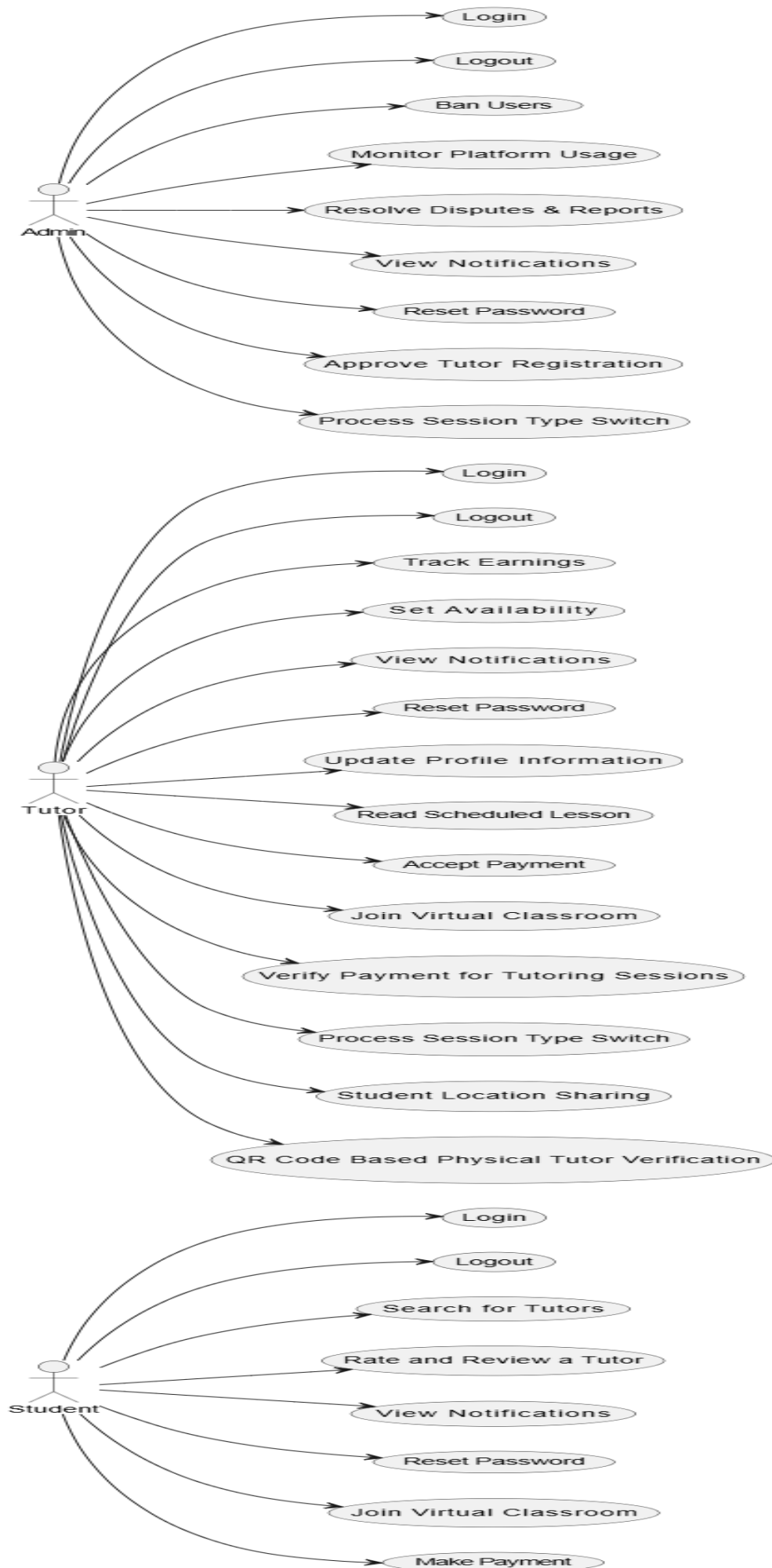
2.4.1 Essential Use Case Modeling

2.4.1.1 Use Case Diagram

This section shows a high-level diagram that incites the interactions between the main actors (students, tutors, and admin) and the system.

Figure II Use

case Diagram



2.4.1.2 Use Case Documentation

Table 1 Essential use cases

Hourly Payment for Tutoring Sessions							
Usecas e id	Usecase name	actor s	description	precondition	Basic flow	Alternative flow	Post condition
Euc-001	Hourly Payment for Tutoring Sessions	Student	This use case allows students to pay tutors based on the duration of the tutoring session session.	1. The student has an active account on the platform. 2. The tutor and student have agreed on the session duration. 3. The system is integrated with a payment gateway for transaction processing. 4. The tutor has set available hours for booking and payment.	1. The student logs into the system using their user ID (UID) and username (UName) 2. The student navigates to the "Book Session" section and selects the tutor. 3. The student selects the desired session duration (in hours). 4. The student confirms the booking details, which are based on the agreed-upon hourly rate. 5. The student proceeds to the payment screen and selects a payment method 6. The student completes the payment.	1 Payment Failure Condition: Payment transaction fails due to insufficient funds, network issues, or gateway errors. Actor Actions: 1. The student receives an error message about the failed payment	1. Payment is successfully processed, and the student's balance is updated for the corresponding session(s). 2. A confirmation notification is sent to both the student and the tutor. 3. The system updates the session record with details about the paid hours.

Process Session Type Switch							
Usecas e id	Usecase name	actors	description	precondition	Basic flow	Alternative flow	Post condition
Euc-002	Process Session Type Switch	Student and tutor	This use case enables the switching of session types during an ongoing tutoring session. Tutors or students can change the session format. The system updates the session details accordingly.	1. A session has been booked and is awaiting execution. 2. A switch request has been initiated by the student or tutor.	1. The system receives the session type switch request from either client. 2. It notifies the other client of the request. 3. Upon their approval, the system updates session type details. 4. Both parties are notified of the updated session type.	If the either clients rejects the switch: 1. The system notifies the another client of the rejection. 2. No changes are made to the session type.	1. Session type details are updated (if approved). 2. Notifications reflect the session type change.

Login							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition
Euc-004	Login	Student, Tutor, Admin	This use case allows users to access their accounts by entering their credentials (username and password).	The user must have valid login credentials	1. The user enters their email/username and password. 2. The system verifies the credentials. 3. The user is granted access to the system.	1. If the credentials are incorrect, an error message is displayed. 2. The user can reset their password using the "Forgot Password" option.	The user is logged into the system.
Logout							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition
Euc-005	Logout	Student, Tutor, Admin	This use case allows users to securely exit their accounts by terminating the session.	The user must be logged in.	1. The user selects the "Logout" option. 2. The system terminates the session.	if session termination error occurred: the system display error and recommend to try again.	The user is logged out of the system.
Search for Tutors							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition

Euc-006	Search for Tutors	Student	This use case allows students to search for available tutors based on specific criteria such as subject, availability, and ratings.	The student must be logged in.	1. The student enters search criteria (e.g., subject, availability). 2. The system retrieves matching tutor profiles. 3. The student views tutor details.	1. If no tutors match the criteria, a message is displayed with alternative suggestions.	The system displays a list of suitable tutors.
Rate and Review a Tutor							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition
Euc-007	Rate and Review a Tutor	Student	This use case allows students to provide feedback on their tutoring experience by rating and reviewing the tutor.	The student must have at least one completed lesson with the tutor (in-person or online).	1. The student selects the tutor from their history. 2. The system presents a rating and review interface. 3. The student submits	1. If the class is rating is not submitted, the system prompts the user to complete it later	The feedback is recorded and reflected in the tutor's profile

					feedback.		
Ban Users							
Usecas e id	Usecase name	actors	description	preconditio n	Basic flow	Alternative flow	Post condition
Euc-008	Ban Users	Admin	This use case allows administrators to ban users who violate platform policies..	1. The user must have an active account. 2. There must be a valid reason for banning the user (e.g., violation of terms).	1. The admin navigates to the "User Management" section. 2. The system displays a list of active users. 3. The admin selects a user and specifies a reason for banning. 4. The system validates the action and bans the user. 5. The user is notified of the ban.	1. If the admin cancels the process, the user remains active.	The user is banned, and access to the platform is revoked
Monitor Platform Usage							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition

Euc-009	Monitor Platform Usage	Admin	Monitor Platform Usage: This use case allows administrators to track and analyze user activity on the platform.	The admin must have valid login credentials and access permissions	1. The admin navigates to the "Platform Analytics" dashboard. 2. The system displays usage metrics such as active users, total sessions, payment history, and lesson statistics. 3. The admin selects a specific metric for detailed analysis. 4. The system displays the detailed report based on the admin's selection.	1. If no data is available (e.g., no users or payments), the system displays an empty message. 2. If load error existed The system fails to load the analytics, and an error is logged	The admin successfully views the platform usage metrics.
Resolve Disputes & Report							
Usecase id	Usecase name	actors	description	precondition	Basic flow	Alternative flow	Post condition

Euc-010	Resolve Disputes & Report	Admin	This use case allows Admin to check report issues and resolve disputes.	A dispute must be reported by a student or tutor.	1. The admin navigates to the "Disputes" section. 2. The system displays a list of disputes. 3. The admin selects a dispute and views its details. 4. The admin takes action (e.g. ban a user) and closes the dispute.	If the database update fails, the system logs the issue and informs the admin.	The dispute is resolved, and both parties are notified.
Track Earnings							
Usecase id	Usecase name	actors	description	precondition	Basic flow	Alternative flow	Post condition
Euc-011	Track Earnings	Tutor	This use case allows tutors to monitor their earnings from tutoring sessions.	The tutor must have received at least one payment	1. The tutor navigates to the "Earnings" section. 2. The system displays a summary of total earnings, pending payments, and payment history. 3. The tutor selects a payment record to view detailed information.	. If no payment records exist, the system displays a "No Payments Found" message.	The tutor views their earnings summary.
Set Availability							
Usecase id	Usecase name	actors	description	precondition	Basic flow	Alternative flow	Post condition

Euc-012	Set Availability	Tutor	This use case allows tutors to specify their available time slots for tutoring sessions.	The tutor must have an active account.	1. The tutor navigates to the "Availability" section in their profile. 2. The system displays a calendar interface. 3. The tutor selects available dates and time slots and submits them. 4. The system validates the input and updates the tutor's schedule.	. If a selected slot overlaps with an existing booking, the system notifies the tutor and prompts for adjustment.	The availability is updated, and students can view the new slots.
View Notifications							
Usecas e id	Usecase name	actors	description	preconditio n	Basic flow	Alternative flow	Post condition
Euc-013	View Notifications	Student, Tutor, Admin	This use case allows users to view important updates and alerts, such as session reminders, messages, or platform announcements.	The user must have at least one notification generated by the system.	1. The user navigates to the "Notifications" section. 2. The system retrieves and displays the list of notifications. 3. The user selects a notification to view its details.	. If no notifications exist, the system displays a "No Notifications" message.	The user views their notifications.
Reset Password							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition

Euc-014	Reset Password	Student, Tutor, Admin	This use case allows users to recover or change their password if forgotten or compromised.	The user must have a secret question registered in the system	1. The user clicks "Forgot Password" on the login page. 2. The system prompts the user to enter their secret questions. 3. The user submits the text, and the system display a reset link	1. If the answer is not registered, the system notifies the user and prompts to retry.	The password is updated, and the user can log in with the new password.
Update Profile Information							
Usecas e id	Usecas e name	actors	description	precondit ion	Basic flow	Alternative flow	Post condition
Euc-015	Update Profile Information	Student, Tutor, Admin	This use case allows users to modify their personal details, such as name, contact information, and preferences.	The user must be logged into their account.	1. The user navigates to their profile section. 2. The system displays editable fields (e.g., name, email, phone number, password). 3. The user modifies the desired information and submits the changes. 4. The system validates the data and confirms the update.	. If the submitted data is invalid (e.g., invalid email format), the system prompts the user to correct it.	The profile is updated successfully, and the user sees a confirmation message.
Reset Scheduled Lesson							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition

Euc-016	Reset Scheduled Lesson	Student, Tutor	This use case allows users to cancel or reschedule a previously set tutoring session.	A scheduled lesson must exist	1. The user selects the lesson from their schedule. 2. The system displays a rescheduling prompt. 3. The user make updates and confirms the rescheduling. 4. The system updates the schedule and notifies the other party.	If cancellation fails, the system logs the error and notifies the user.	The lesson is rescheduled, and both parties are notified.
Join Virtual Classroom							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition
Euc-017	Join Virtual Classroom	Student	This use case allows students and tutors to enter an online session through a virtual classroom link.	The student must have scheduled a lesson and received a virtual classroom link	1. The student logs in and navigates to the "My Lessons" page. 2. The system displays a "Join Now" button for the scheduled class. 3. The student clicks the button, and the system redirects them to the virtual classroom.	1. If the session hasn't started, the system displays a countdown timer. 2. If the virtual classroom link is invalid, an error message appears.	The student joins the live session.
Make Payment							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition

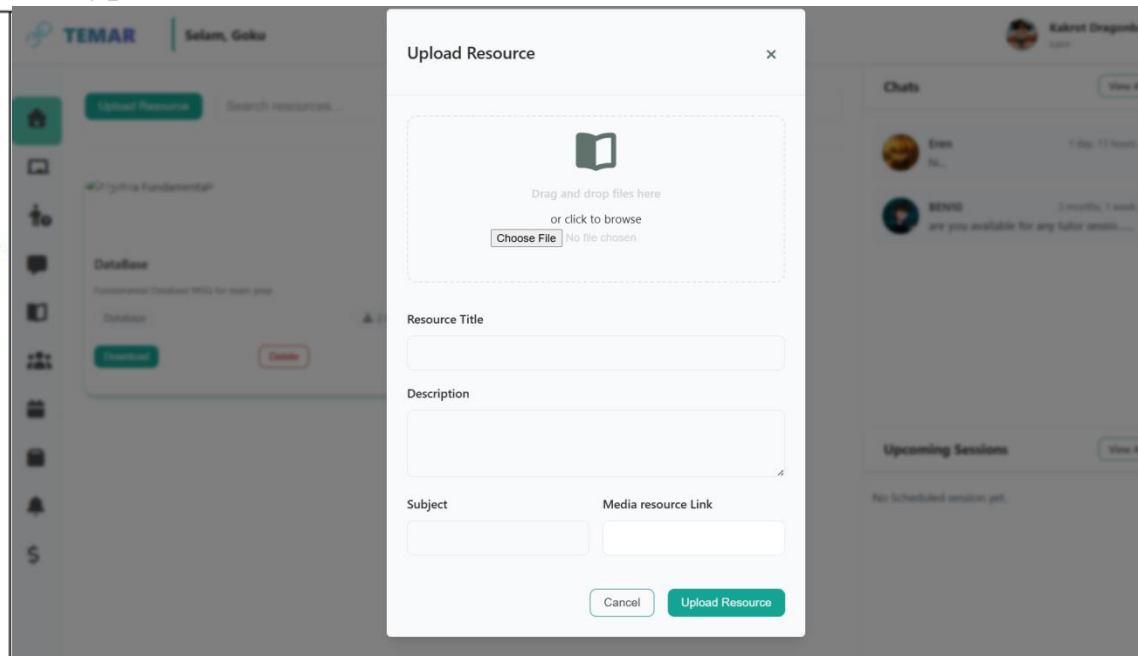
Euc-018	Make Payment	Student	This use case allows students to pay for tutoring sessions through a secure payment gateway..	The student must have a valid payment method.	1. The student selects a tutor and lesson package. 2. The system calculates the total cost, including any fees. 3. The student confirms the payment. 4. The system processes the payment through a payment gateway.	If the payment is declined, the student is prompted to retry or use another method.	The payment is successful, and the system notifies both the student and the tutor.
Accept Payment							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition
Euc-019	Accept Payment	tutor	This use case allows tutors to receive payments for their tutoring sessions.	A student must have made a payment for a lesson.	1. The tutor logs in and navigates to the "Payments" page. 2. The system displays the payment details. 3. The tutor clicks "Accept Payment," and the system updates the tutor's account balance.	If any system error occurred the system display error message	The payment is added to the tutor's earnings.
Host Virtual Classroom							
Usecas e id	Usecase name	actors	description	precondi tion	Basic flow	Alternative flow	Post condition

Euc-020	Host Virtual Classroom	tutor	This use case allows tutors to create and manage an online tutoring session..	The tutor must have scheduled a session	1. The tutor logs in and navigates to "My Classes." 2. The system displays upcoming sessions. 3. The tutor clicks "Start Class," and the virtual classroom launches.	1. If the session fails to start, the system displays troubleshooting options.	The virtual classroom is live.
Approve Tutor Registration							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
Euc-021	Approve Tutor Registration	Admin	This use case allows administrators to review and approve tutor registrations.	Tutors must have submitted registration details.	1. The admin logs in and navigates to "Pending Registrations." 2. The system lists unapproved tutor profiles. 3. The admin reviews and approves/rejects the profile.	1. If the admin rejects a profile, a rejection reason is sent to the tutor.	The tutor is approved and visible to students

2.4.2 Essential UI Prototype

Material Sharing Window

This window enables tutors to upload, organize, and share educational materials such as PDFs, videos, and notes.



Window for sharing Educational Materials

Chat Window

This interface allows real-time messaging between students and tutors

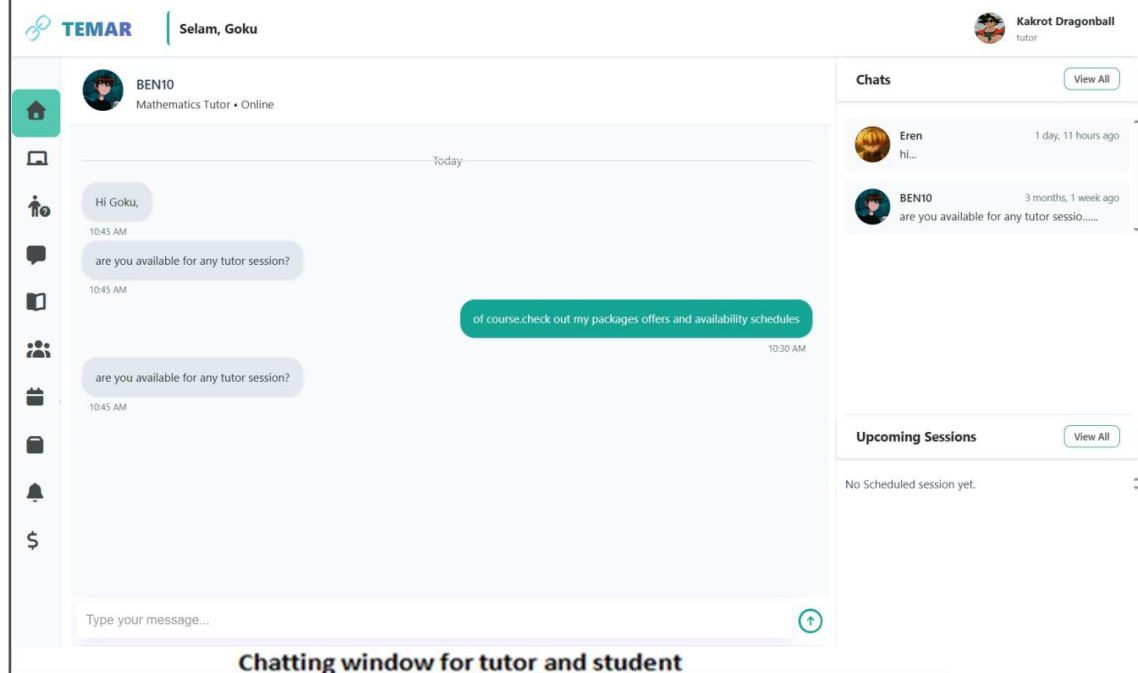


Figure III UI prototype

Tutor profiles are displayed with essential details, including qualifications, reviews, and booking options, allowing students to find the best match for their learning needs.

Tutor profiles are displayed with essential details, including qualifications, reviews, and booking options, allowing students to find the best match for their learning needs.

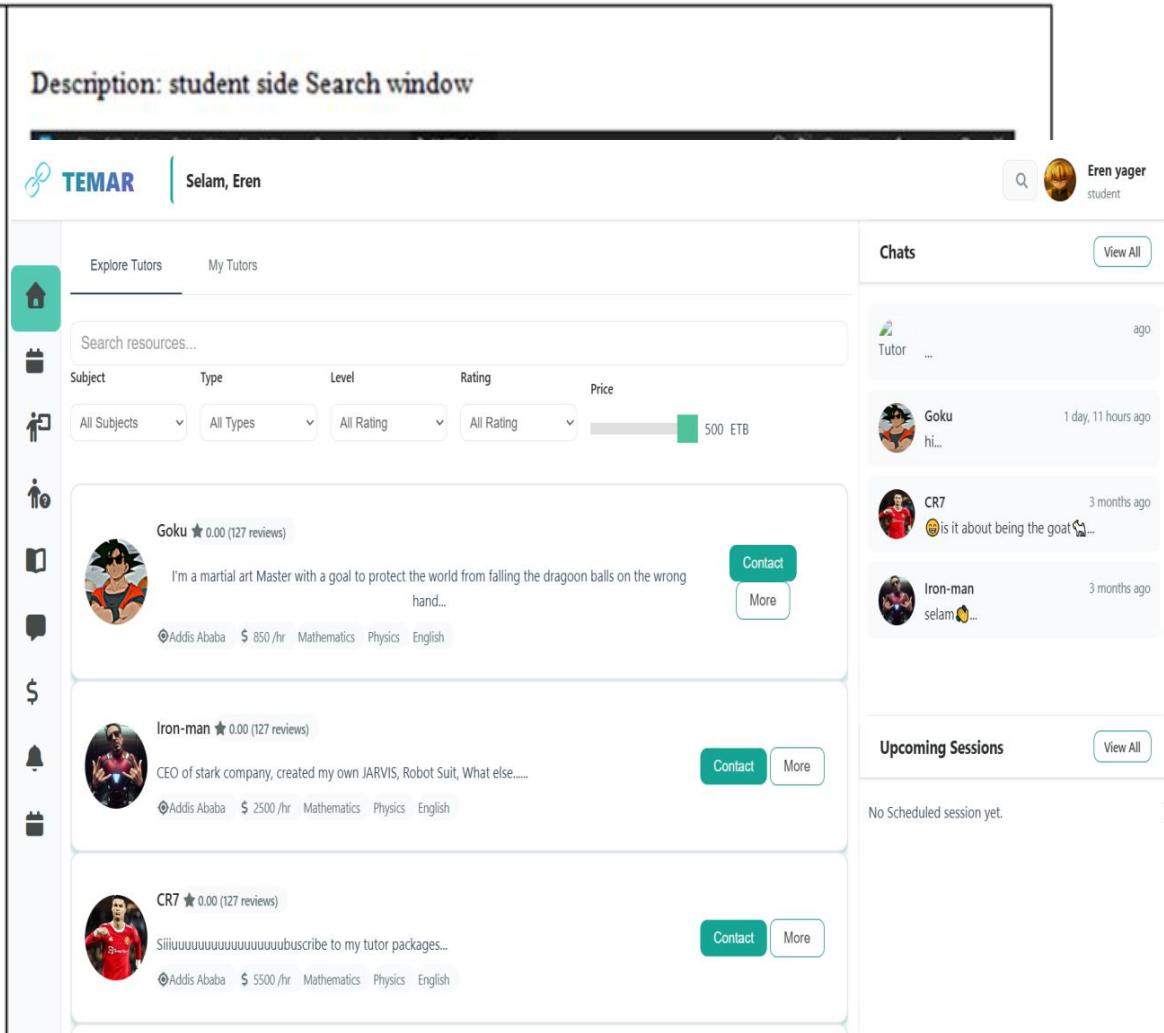


Figure IV UI prototype

Table 2 User interfaces

Login Page			
ID	NAME	ELEMENT	DISCRIPTION
UI-001	Login Page	Username Field , Password Field , Login Button , Forgot Password Link , Sign-Up Link	Text input field for entering the username or email address. Text input field for entering the user's password. Button to submit the login credentials for authentication. Redirects to a page for password recovery. Redirects to the registration page for new users.
Registration Page			
ID	NAME	ELEMENT	DISCRIPTION
UI-002	Registration Page	Full Name Field , Password Field, Confirm Password Field, Role Selection, Dropdown, Register Button	Text input field for entering the user's full name. Text input field for entering a valid email address. Text input field for creating a secure password. Text input field for confirming the entered password. Dropdown menu to select between roles like Student, Tutor, or Admin. Button to submit the registration form.
Dashboard (Common Interface))			

ID	NAME	ELEMENT	DISCRIPTION
UI-003	Dashboard (Common Interface))	Navigation Bar , Notifications Icon , Search Bar , Quick Actions	Includes options like Home, Profile, Messages, and Logout. Displays the count of unread notifications and redirects to the notification list. Text input for searching tutors, students, or lessons. Buttons or links for tasks like booking lessons, managing users, or sending notifications.
Search for Tutors (Student Interface)			
ID	NAME	ELEMENT	DISCRIPTION
UI-004	Search for Tutors (Student Interface)	Search Bar , Filter Options , Tutor Profile Cards , View Profile Button	Text input field to enter tutor names, subjects... Filters for narrowing results by subject, rating, or price. Displays a preview of tutor details such as name, expertise, rating, and price. Button to view detailed tutor profiles.
Tutor Profile Page			
ID	NAME	ELEMENT	DISCRIPTION
UI-005	Tutor Profile Page	Profile Picture ,	Displays the tutor's profile

		Name and Bio Section , Ratings and Reviews , Contact/Book Button	photo. Shows the tutor's name and brief biography. Displays average rating and user reviews. Button to initiate booking or communication with the tutor.
Lesson Scheduling Page(student)			
ID	NAME	ELEMENT	DISCRIPTION
UI-006	Lesson Scheduling Page(student)	Calendar View , Time Slot Selection Dropdown , Confirm Booking Button	Displays available dates and time slots for lesson booking. Dropdown menu to select a specific time slot. Button to finalize and confirm the lesson booking.
Admin Dashboard			
ID	NAME	ELEMENT	DISCRIPTION
UI-007	Admin Dashboard	User Management Table , Notifications Panel , System Reports	Displays a list of users with options to edit or delete user profiles. Allows the admin to send system-wide notifications. Displays analytics on user activity, lesson bookings, and payments.
Rate and Review Interface			
ID	NAME	ELEMENT	DISCRIPTION
UI-008	Rate and Review Interface	Rating Slider/Stars Review Text Field Submit Button	Allows users to provide a star rating for tutors.

			Text input field for users to write detailed feedback. Button to save and publish the review.
Payment Interface			
ID	NAME	ELEMENT	DISCRIPTION
UI-009	Payment Interface	Payment Amount Field, Payment Method Dropdown , Confirm Payment Button , Payment History Table	Displays the total payment amount. Dropdown menu to choose payment methods like credit card, mobile payment, etc. Button to finalize the payment. Displays a record of previous transactions.
Notification Page			
ID	NAME	ELEMENT	DISCRIPTION
UI-010	Notification Page	Notification List , Mark as Read Button	Displays all unread and read notifications. Button to mark specific notifications as read.
Logout Interface			
ID	NAME	ELEMENT	DISCRIPTION
UI-011	Logout Interface	Confirmation Dialog , Logout Button	Dialog box to confirm the logout action. Button to finalize logout and redirect to the login page.
Forgot Password Page			
ID	NAME	ELEMENT	DISCRIPTION

UI-012	Forgot Password Page	Email Input Field , Submit Button , Back to Login Link	Text input field where users enter their registered email to reset the password. Button to send a password reset link to the entered email. Redirects the user back to the login page.
Virtual Classroom Interface			
ID	NAME	ELEMENT	DISCRIPTION
UI-013	Virtual Classroom Interface	Video/Audio Stream Window , Chat Box , End Session Button	Main window for live video or audio streaming during the lesson. Allows participants to send text messages during the session. Button to end the current session.
Profile Management Interface			
ID	NAME	ELEMENT	DISCRIPTION
UI-014	Profile Management Interface	Edit Profile Button , Profile Picture Upload , Save Changes Button , Cancel Button	Button to enable editing of profile details. Option to upload or update the profile picture. Button to save updated profile details. Button to discard changes and revert to the previous profile

			state
Lesson Reschedule Page			
ID	NAME	ELEMENT	DISCRIPTION
UI-015	Lesson Reschedule Page	Reschedule Reason Dropdown, Additional Comments Text Field , Reschedule selection Calendar, Confirm Reschedule Button , Notification Message	Dropdown for selecting the reason for Rescheduling the lesson. Optional field to provide further explanation for Rescheduling. Calendar for select reschedule date. Button to finalize and submit the Rescheduling request. Displays a confirmation message that the lesson was Rescheduled successfully.
Report Dispute Interface			
ID	NAME	ELEMENT	DISCRIPTION
UI-016	Report Dispute Interface	Dispute Reason Dropdown , Dispute Description Field , Submit Dispute Button, View Status Button	Dropdown menu for selecting the reason for disputing . Text field to provide a detailed explanation of the dispute. Button to submit the dispute for review. Option to check the status of the submitted dispute.
Feedback and Support Page			
ID	NAME	ELEMENT	DISCRIPTION

UI-017	Feedback and Support Page	Feedback Text Field , Submit Feedback Button , Support Ticket Form , View Ticket Status	Text area where users can provide their feedback or suggestions about the platform. Button to send the feedback to the system admin. Form to create a support ticket for issues or queries. Option to track the status of submitted support tickets.
Tutor Availability Management Interface			
ID	NAME	ELEMENT	DISCRIPTION
UI-018	Tutor Availability Management Interface	Calendar View , Add Availability Button, Remove Availability Button , Save Changes Button	Displays the tutor's availability schedule for lessons. Allows tutors to specify available dates and time slots. Option to delete or adjust availability. Button to save updates to the availability schedule.
Tutors Verification Page (Admin Only)			
ID	NAME	ELEMENT	DISCRIPTION
UI-019	Tutors Verification Page (Admin Only)	Pending Verification Table , Approve Button , Reject Button , Verification Filters	Displays a list of unverified Tutors. Allows admins to verify and

			approve a Tutor profile. Allows admins to reject an incomplete or invalid profile. Filters to view Tutors by registration date, or status
Personalized Dashboard (Student and Tutor Specific)			
ID	NAME	ELEMENT	DISCRIPTION
UI-020	Personalized Dashboard (Student and Tutor Specific)	Upcoming Lessons Widget , Recent Notifications , Quick Access Links	Displays a list of upcoming lessons with date, time, and tutor/student details. Shows the latest notifications related to bookings, payments, or updates. Provides shortcuts to common tasks like scheduling a lesson, managing a profile, or checking payment history.
Admin Notification Management Interface			
ID	NAME	ELEMENT	DISCRIPTION
UI-021	Admin Notification Management Interface	Compose Notification Button , Select Recipients Dropdown , Notification History	Option for admins to create a new system-wide notification. Allows selection of notification recipients (e.g., all users, tutors only, etc.). Displays a list of previously sent notifications with timestamps.
Lesson History Page (For Students and Tutors)			
ID	NAME	ELEMENT	DISCRIPTION
UI-022	Lesson History Page (For	Lesson History Table ,	Displays all completed lessons

	Students and Tutors)	Filter by Date/Subject , Rebook Lesson Button	with details like date, time, tutor/student name, and subject. Allows users to filter lessons by date range or subject type. Provides an option to quickly rebook a completed lesson with the same tutor/student.
Payment Withdrawal Page (For Tutors)			
ID	NAME	ELEMENT	DISCRIPTION
UI-023	Payment Withdrawal Page (For Tutors)	Account Balance Display , Withdrawal Method Dropdown , Amount Input Field , Submit Withdrawal Request Button , Withdrawal History Table	Shows the tutor's available balance for withdrawal. Allows tutors to select a preferred withdrawal method (e.g., bank transfer, mobile payment). Text input for entering the amount to withdraw. Button to process the withdrawal request. Displays a record of past withdrawal requests and their statuses.
Lesson Reminder Notification Interface			
ID	NAME	ELEMENT	DISCRIPTION

UI-024	Lesson Reminder Notification Interface	Notification Settings , Reminder Preview	Allows users to enable or disable reminders for upcoming lessons. Displays details of the next lesson (e.g., date, time, tutor/student name).
Lesson Feedback and Rating Interface (For Students)			
ID	NAME	ELEMENT	DISCRIPTION
UI-025	Lesson Feedback and Rating Interface (For Students)	Rating Stars , Feedback Text Area , Submit Feedback Button , Confirmation Message	Allows students to rate the lesson on a scale of 1 to 5 stars. Text input where students can provide written feedback about the lesson Button to submit the feedback and rating for the lesson. Displays a message confirming the successful submission of feedback.
Tutor Profile Search Interface (For Students)			
ID	NAME	ELEMENT	DISCRIPTION
UI-026	Tutor Profile Search Interface (For Students)	Search Bar , Filter Options , Tutor Profile Card , View Profile Button	A text input field for searching tutors by name, subject, or rating. Filters to narrow down the search by subject, price, rating, or availability. A preview card for each tutor,

			showing their photo, name, rating, and subjects offered. Button to view detailed information about a specific tutor.
Admin User Management Page			
ID	NAME	ELEMENT	DISCRIPTION
UI-027	Admin User Management Page	User List , Search Bar , User Button , Block/Unblock User Button , Delete User Button	Displays all registered users (students and tutors) with their details (name, email, role, status). Allows admins to search for specific users by name or email. Option for the admin to modify user details (e.g., role, status). Allows the admin to block or unblock a user account. Option to delete a user's account from the system.
Admin Payment Management Page			
ID	NAME	ELEMENT	DISCRIPTION

UI-028	Admin Payment Management Page	Payment Summary Table , Filter by Date/Amount/Status	Displays all payment transactions made on the platform with details like payment date, amount, and status. Filters for reviewing payment history by date range, payment amount, or status.
Terms and Conditions Page			
ID	NAME	ELEMENT	DISCRIPTION
UI-029	Terms and Conditions Page	Terms Text , Agree to Terms Checkbox , Submit Button	Displays the terms and conditions of using the platform. Checkbox for users to accept the terms and conditions before proceeding. Button to submit the form after accepting the terms.
Material Sharing Page			
ID	NAME	ELEMENT	DISCRIPTION
UI-030	Material Sharing Page	Upload Material Button, Submit Button, Cancel Button	A button to upload materials (PDF, videos, images). A button to finalize material sharing. A button to discard changes.
Material Access Page			

ID	NAME	ELEMENT	DISCRIPTION
UI-031	Material Access Page	Material List, Download Button, Search Bar	A list of available materials with file name, upload date, tutor name, and access status. A button to download materials. A search bar to find specific materials by tutor name or file name.
Lesson Mode Switcher			
ID	NAME	ELEMENT	DISCRIPTION
UI-032	Lesson Mode Switcher	Toggle Switch, Confirmation Popup, Submit Button, Cancel Button	A toggle to switch between "Physical" and "Virtual" modes. A popup to confirm the switch, including new lesson mode details. A button to confirm the mode switch. A button to cancel the operation.
Schedule Availability Checker			
ID	NAME	ELEMENT	DISCRIPTION

UI-033	Schedule Availability Checker	Availability List, Approve Button, Reject Button, Filter Options	A list of available slots with student UID, UName, and requested time. A button to approve the scheduling request. A button to reject the request. Filters to sort by date or student name.
--------	-------------------------------	---	--

2.4.3 User Interface Flow Diagram

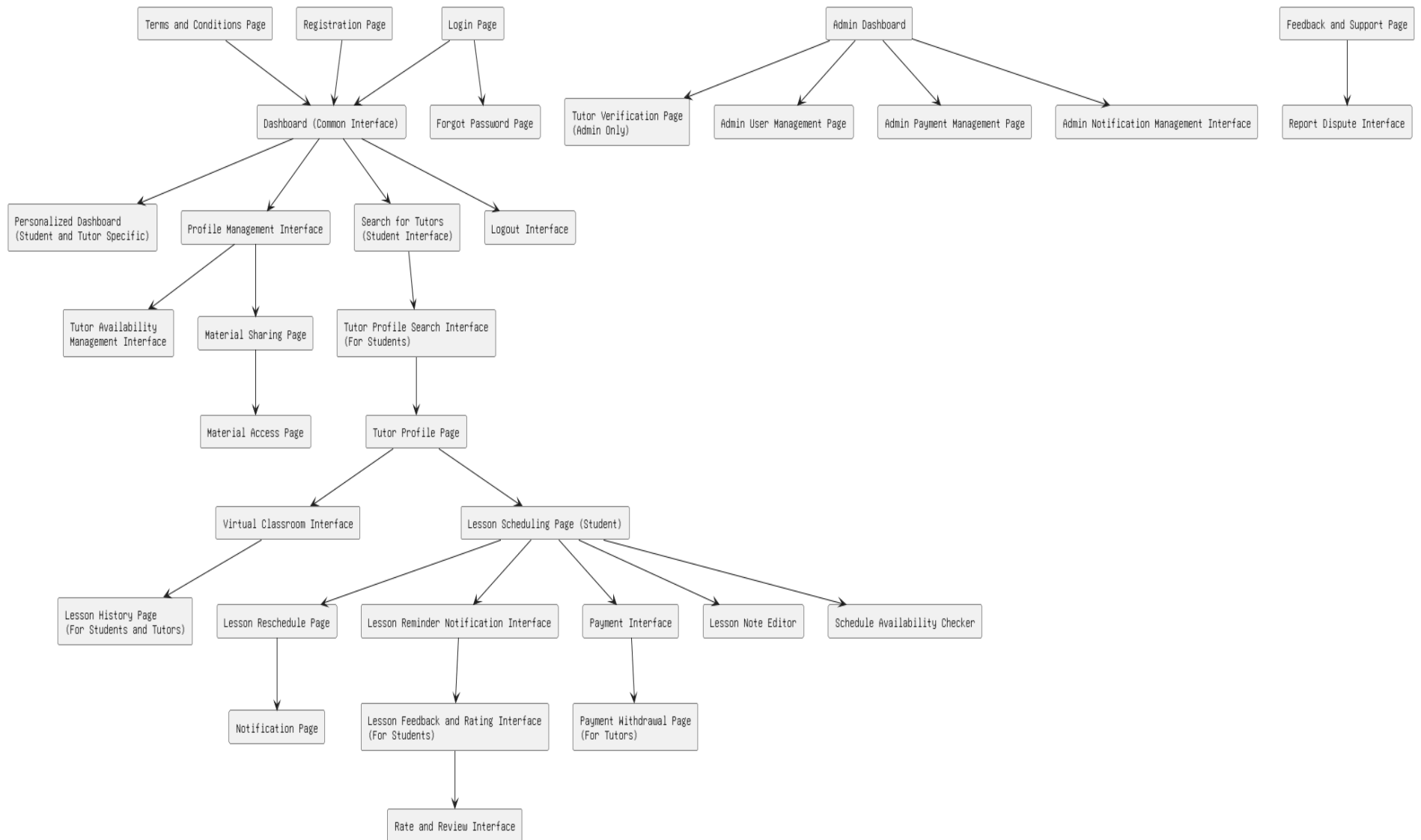


Figure V user interface flow diagram

2.4.4 Supplementary Specifications

2.4.4.1 Business Rules

Table 3 business Rules

Id	description	priority	dependency	Potential impact
BR-1	Users must register using verified user name addresses and phone numbers.	High	none	Ensures valid user data, preventing spam or fake accounts.
Id	description	priority	dependency	Potential impact
BR-2	User passwords must meet complexity requirements (e.g., minimum 8 characters, at least one symbol, number, and uppercase letter).	High	BR-1	Enhances account security and reduces vulnerability to brute force attacks.
Id	description	priority	dependency	Potential impact
BR-3	Users must log in with secure credentials, including password	High	BR-2	Enhances platform security and protects user data.
Id	description	priority	dependency	Potential impact
BR-4	Tutors and students must accept terms and conditions during registration.	High	none	Ensures legal compliance and clarifies user responsibilities.
Id	description	priority	dependency	Potential impact
BR-5	All payments must be processed through secure gateways supporting multiple payment methods (e.g., cards, wallets).	High	none	Secures transactions; unreliable payment processing could lead to financial disputes.
Id	description	priority	dependency	Potential impact
BR-6	Payments must be verified before scheduling confirmation.	High	BR-5	Prevents unpaid bookings and ensures reliable transactions.

Id	description	priority	dependency	Potential impact		
BR-7	Platform fees (commission) must be automatically deducted from tutor payments.	High	BR-5	Generates platform revenue and maintains transparency for tutors.		
Id	description	priority	dependency	Potential impact		
BR-8	Disputes must be raised within 7 days of a session and resolved within 14 days.	High	none	Ensures timely resolution and user satisfaction; delays can impact platform trust.		
Id	description	priority	dependency	Potential impact		
BR-9	Tutors must specify their expertise and subjects during registration.	High	BR-1	Helps students find tutors who match their specific learning needs.		
Id	description	priority	dependency	Potential impact		
BR-10	Physical sessions must include time tracking for session management and ease of access	Medium	none	Improves safety and convenience for users;		
Id	description	priority	dependency	Potential impact		
BR-11	Users must have the option to reset their password using secret question verification.	High	BR-1	Improves user experience and prevents account lockouts due to forgotten passwords.		
Id	description	priority	dependency	Potential impact		
BR-12	Platform must support automated evidence collection for disputes, such as session logs and chats.	Medium	BR-8	Speeds up dispute resolution and provides unbiased evidence.		
Id	description	priority	dependency	Potential impact		
BR-13	Tutors and students must provide feedback on physical sessions for quality assurance.			Medium	none	Improves service quality and highlights areas for improvement.
Id	description			priority	dependency	Potential impact
BR-14	Admins must have the ability to suspend or flag accounts showing suspicious activity.			High	BR-4	Prevents fraud and ensures platform integrity.
Id	description			priority	dependency	Potential impact

BR-15	System must scale horizontally to handle an increase in users without performance degradation.	High	none	Ensures consistent performance during peak usage periods.
Id	description	priority	dependency	Potential impact
BR-16	Tutors must undergo periodic performance reviews based on student feedback and session quality.	Medium	none	Improves overall tutor quality and user satisfaction.
Id	description	priority	dependency	Potential impact
BR-17	Physical session check-in and check-out times must be logged for accurate billing.	High	BR-4	Ensures fairness and prevents payment disputes.
Id	description	priority	dependency	Potential impact
BR-18	session Reschedules must be notified at least 24 hours in advance to avoid penalties.	High	BR-4	Encourages commitment and reduces last-minute cancellations.
Id	description	priority	dependency	Potential impact
BR-19	Tutors and students must have access to a chat history log to reference in disputes.	Medium	BR-8	Provides evidence for resolving conflicts and improves transparency.
Id	description	priority	dependency	Potential impact
BR-20	Physical sessions must allow students to reschedule up to 3 times only.	High	BR-4	Provides flexibility for students while maintaining session accountability.

2.4.4.2 Constraints

- Platform accessible only on modern web browsers.
- Payments restricted to verified methods (e.g., credit cards, PayPal).

2.4.4.3 Change Case

- Tutors may change availability up to 24 hours before a session.
- Students may reschedule with prior notice within 24 hours before session.

2.5 System Models - Analysis

2.5.1 System Use Case Modeling

2.5.1.1 Use Case Diagram

A detailed system use case diagram, including subsystems like authentication, tutor management, and session scheduling.

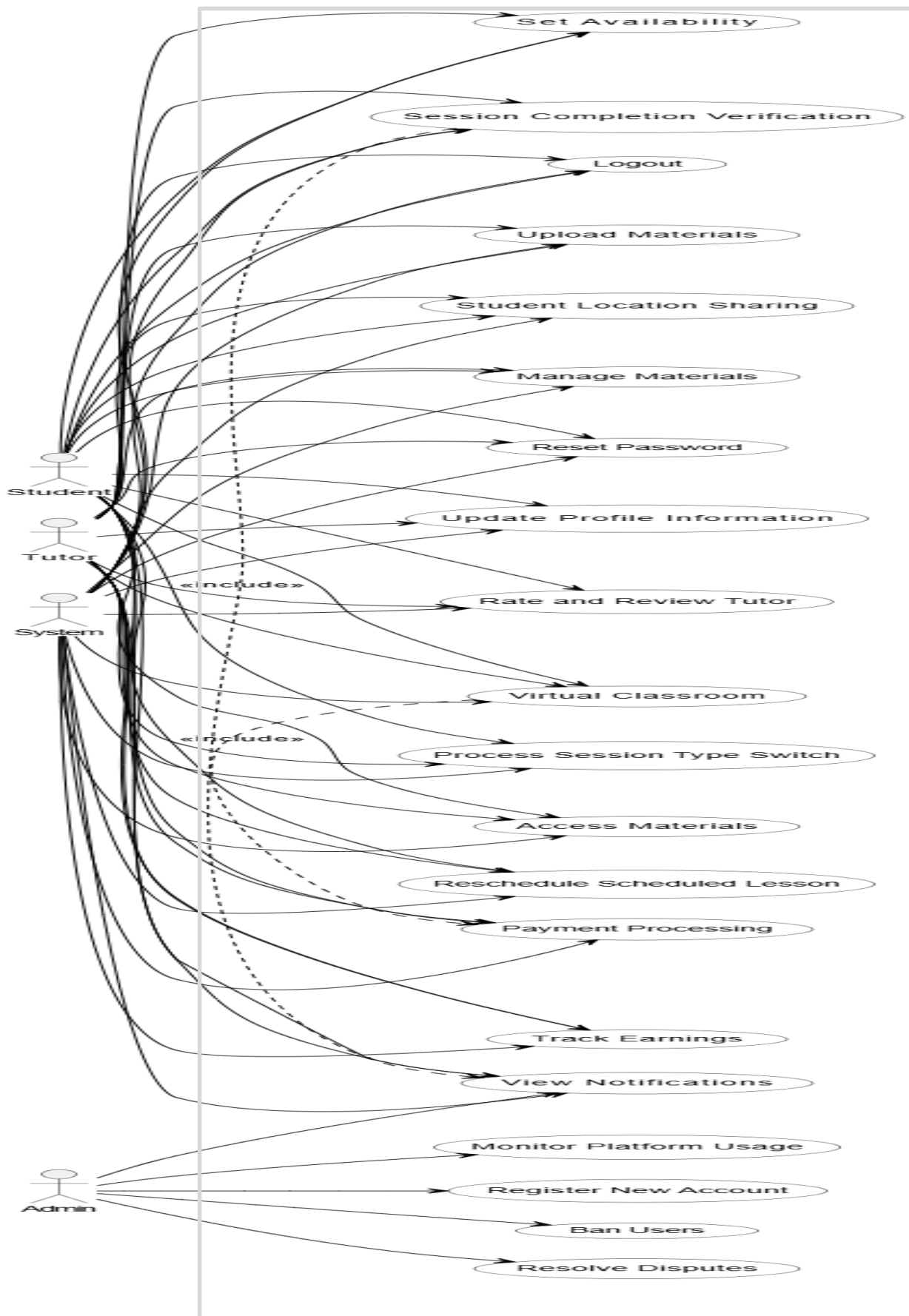


figure VI Use Case diagram

2.5.1.2 Use Case Documentation

Detailed documentation for key system use cases like "Manage Tutor Profiles" or "Process Payments."

Table 4System use cases

Access Materials							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-001	Access Materials	Student, system	This use case allows an authenticated user to retrieve and view digital educational content	1.Student is logged in. 2.Materials have been shared with the student.	1. Student navigates to "Shared Materials" section. 2. System displays a list of materials associated with the student. 3. Student selects a material. 4. System fetches and displays the material. 5. Student views or downloads the material.	if fetching error occurred display error and suggest to try again	Material is accessed successfully.

Manage Materials							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-002	Manage Materials	Tutor, system	This use case allows an authorized user to create, update, or delete educational materials	1.Tutor is logged in. 2.Materials uploaded by the tutor exist.	1. Tutor navigates to "Material Sharing" section. 2. System displays a list of uploaded materials. 3. Tutor selects a material. 4. Tutor updates the description or visibility. 5. System validates changes and saves updates. 6. Alternatively, the tutor deletes the material. 7. System removes the material from the database and notifies affected students.	if error occurred during update, display error and suggest try again.	Material is updated or removed.
Upload Materials							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-003	Upload Materials	Tutor, system	This use case allows an authorized user to upload new educational materials	1.Tutor is logged in. 2.Tutor is associated with at least one student	1. Tutor navigates to "Material Sharing" section. 2. Tutor selects a student. 3. Tutor uploads a file. 4. System validates the file type and size. 5. Tutor adds a description and sets visibility. 6. System saves the material to the database 7. System notifies the selected student(s).	if applied error occur, the system display occur and suggest to retry.	Material is available to selected students

Process Session Type Switch							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-004	Process Session Type Switch	System, tutor, student	This use case processes a change between session types by validating the request, verifying permissions, and updating the session parameters accordingly.	1. A session has been booked and is awaiting execution. 2. A switch request has been initiated by the either party(student or tutor).	1. The system receives the session type switch request. 2. It notifies the other party of the request. 3. Upon approval, the system updates session type details. 4. Both parties are notified of the updated session type.	If the other party rejects the switch: 1. The system notifies the student of the rejection. 2. No changes are made to the session type	1. Session type details are updated 2. Notifications reflect the session type change.
Manage Payment and Escrow							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition

suc-006	Manage Payment and Escrow	system	This use case enables system to manage financial transactions by processing payments, handling escrow funds, and ensuring secure, verified transfers.	1. The student has completed payment for a session or package. 2. Funds are held in escrow by the platform.	1. The system confirms receipt of the payment. 2. It holds the funds securely in escrow. 3. After session completion, the system releases the funds to the tutor.	if any dispute are issued with session the payment is hold in scow until resolved.	1. Payment is securely handled and tracked in the system. 2. Funds are released to the appropriate party based on conditions.
Session Completion Verification							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-007	Session Completion Verification	system, tutor, student	This use case verifies that a session has met all completion criteria, updates its status, and logs the final outcome.	1. The session has been conducted.	1. For virtual sessions: The system auto-verifies completion based on activity logs or confirms completion with the student. 2. For in-person sessions: The system marks the session as completed based on manual confirmation by the tutor or student.	If a dispute arises: 1. The system requests evidence from both parties. 2. It evaluates the evidence and resolves the dispute	The session is marked as completed or flagged for further review.
Virtual Classroom							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition

suc-008	Virtual Classroom	System, student tutor, conferencing API	This use case facilitates live, interactive learning sessions by enabling teachers and students to engage via real-time audio, video, and shared resources.	1. Scheduled session must exist. 2. Valid API credentials for the video conferencing platform.	1. The system sends a request to the video conferencing API to create a session. 2. The API returns a session link and credentials. 3. The system sends the session details to both student and tutor. 4. The system tracks session activity (start time, duration).	1. If the API is unavailable, the system retries or notifies the admin and client.	Both actors join the session.
Payment Processing							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-009	Payment Processing	system, tutor, student, payment API	This use case handles secure payment transactions by verifying details, processing funds, and confirming payment completion.	Valid payment methods must be linked to the student and tutor	1. The system calculates the payment amount. 2. The student confirms the payment. 3. The system sends the payment request to the gateway. 4. The gateway processes the payment and returns a confirmation. 5. The system take commission and updates the tutor's earnings and notifies both parties.	If the payment fails, the system logs the issue and notifies the client and suggest alternative action.	Payment processed and earnings updated.
Logout							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition

suc-010	Logout	Tutor, admin student, tutor	This use case securely ends a user's session by verifying the logout request, terminating access, and clearing session data.	Active user session.	1. User selects "Logout." 2. The system terminates the session and clears authentication tokens. 3. The system redirects the user to the login page.	If the session termination fails, the system logs the issue.	User is logged out and redirected.
reschedule Scheduled Lesson							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-011	reschedule Scheduled Lesson	Student, tutor, system	This use case allows an authorized user to modify the date and time of a scheduled lesson by verifying availability, updating the schedule, and notifying relevant parties.	The lesson must be scheduled.	1. The system retrieves the lesson details. 2. The user confirms update on detail. 3.the system reschedule lesson.	. If the rescheduling fails, the system retries and logs the issue	Lesson is rescheduled, and notifications are sent
Update Profile Information							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition

suc-012	Update Profile Information	System, tutor, student	This use case allows a user to update their profile details by verifying credentials, processing changes, and storing the updated information securely.	1.The user must be logged in.	1. The user submits the profile update request. 2. The system validates the submitted data. 3. The system updates the user's profile in the database. 4. A success message is sent to the user	1. If validation fails, the system logs the issue and displays an error message. 2. If the database update fails, the system logs the error and prompts the user to retry.	The user's profile is updated in the database.
----------------	----------------------------	------------------------	---	-------------------------------	---	---	--

Reset Password

Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-013	Reset Password	System, student tutor	This use case allows a user to reset their password by verifying identity, generating a secure reset link or code, and updating the password.	The database system must be operational.	1. The user submits a password reset request. 2. The system validates secret question. 3. The system sends a unique reset link to the user. 4. The user clicks the link and submits a new password. 5. The system validates the password and updates it in the database	1. If the answer is invalid or unregistered, the system logs the issue and informs the user.	The password is updated in the database.

View Notifications

Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
-------------	---------------	--------	-------------	--------------	------------	------------------	----------------

suc-014	View Notifications	System, student, tutor, admin	This use case allows a user to access and view system-generated notifications in a structured format.	The user must have at least one notification.	1. The user navigates to the "Notifications" section. 2. The system retrieves and displays the list of notifications. 3. The user selects a notification to view its details.	If no notifications exist, the system displays a "No Notifications" message.	The user views their notifications.
----------------	--------------------	-------------------------------	---	---	---	--	-------------------------------------

Rate and Review Tutor

Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-015	Rate and Review Tutor	Student, system	This use case allows a student to rate and review a tutor by submitting feedback, which is stored and displayed for evaluation.	The student must have an active account and a completed lesson record.	1. The student submits the rating and review. 2. The system validates the input and ensures the student is eligible to review the tutor. 3. The system stores the rating and review in the database. 4. The tutor's profile is updated with the new rating and review	If validation fails (e.g., missing fields), the system prompts the student to correct it. 2. If the database update fails, the system logs the issue and informs the student.	The review is stored in the database and visible on the tutor's profile.

Set Availability

Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
--------------------	----------------------	---------------	--------------------	---------------------	-------------------	-------------------------	-----------------------

suc-016	Set Availability	Tutor, system	This use case allows a tutor to define their available time slots by updating the schedule and saving the availability settings.	The tutor must be authenticated	1. The tutor submits their availability schedule. 2. The system validates the input (e.g., no overlapping slots). 3. The system updates the tutor's schedule in the database. 4. The system sends a confirmation message to the tutor.	1. If the database update fails, the system logs the issue and notifies the tutor.	The availability is updated in the system.
----------------	------------------	---------------	--	---------------------------------	---	--	--

Track Earnings

Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-017	Track Earnings	system, tutor	This use case allows a user to monitor and view their earnings by retrieving transaction data and displaying a summary of financial activity.	The tutor must be authenticated, and payment records must exist in the database	Basic Flow: 1. The tutor requests to view their earnings. 2. The system queries the database for the tutor's payment records. 3. The system retrieves and displays the records	1. If the database query fails, the system logs the issue and notifies the tutor.	The earnings are displayed in the user interface.

Resolve Disputes

Use	Use case	actors	description	precondition	Basic flow	Alternative	Post condition
------------	-----------------	---------------	--------------------	---------------------	-------------------	--------------------	-----------------------

case id	name					flow	
suc-018	Resolve Disputes	system, student, admin, tutor	This use case allows users to address and resolve conflicts by submitting claims, reviewing evidence, and providing a final resolution.	The admin must have sufficient permissions to resolve disputes.	1. The admin selects a dispute to review. 2. The system retrieves the dispute details and displays them. 3. The admin submits a resolution. 4. The system updates the dispute status in the database and notifies the involved parties	If the database update fails, the system logs the issue and informs the admin.	The dispute is marked as resolved.
Monitor Platform Usage							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-019	Monitor Platform Usage	System, admin	This use case allows administrators to track and analyze platform activity by collecting usage data and generating reports for monitoring purposes.	The analytics system and database must be operational .	1. The admin requests analytics data. 2. The system retrieves data from the database and processes it in the analytics module. 3. The system generates and displays the analytics reports.	1. If data retrieval fails, the system logs the issue and notifies the admin. 2. If the analytics processing fails, the system displays a generic error.	The analytics are displayed in the dashboard.

Ban Users							
Use case id	Use case name	actors	description	precondition	Basic flow	Alternative flow	Post condition
suc-020	Ban Users	System, admin	This use case allows administrators to suspend a user's access by verifying violations, applying the ban, and updating the user status.	The user's account must exist in the database.	1. The admin requests to ban a user. 2. The system updates the user's status to "banned" in the database. 3. Notifications are sent to the user about the ban.	If the database update fails, the system logs the issue and informs the admin.	The user's status is updated to "banned," and access is revoked
Recommend Tutors							
Use case id	Use case name	actors	precondition	Basic flow	Alternative flow	Post condition	
suc-021	Recommend Tutors	System, students	Tutor and student data must exist in the database	1. The system retrieves the student's preferences and learning history. 2. The recommendation engine processes the data and identifies suitable tutors. 3. The system displays the recommendations to the student	If the recommendation engine fails, the system provides a generic list of tutors	The student receives personalized tutor recommendations	

2.5.2 Sequence Diagram

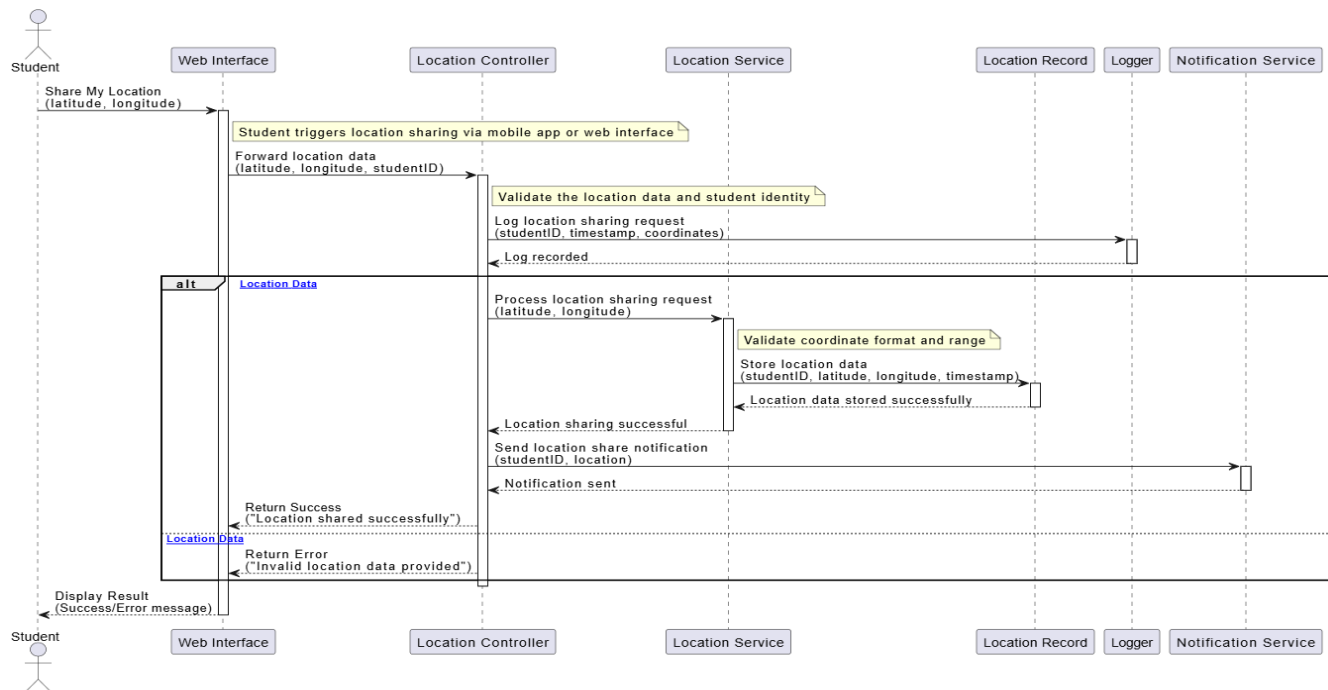


Figure VII Tutor Accept Payment Sequence Diagram

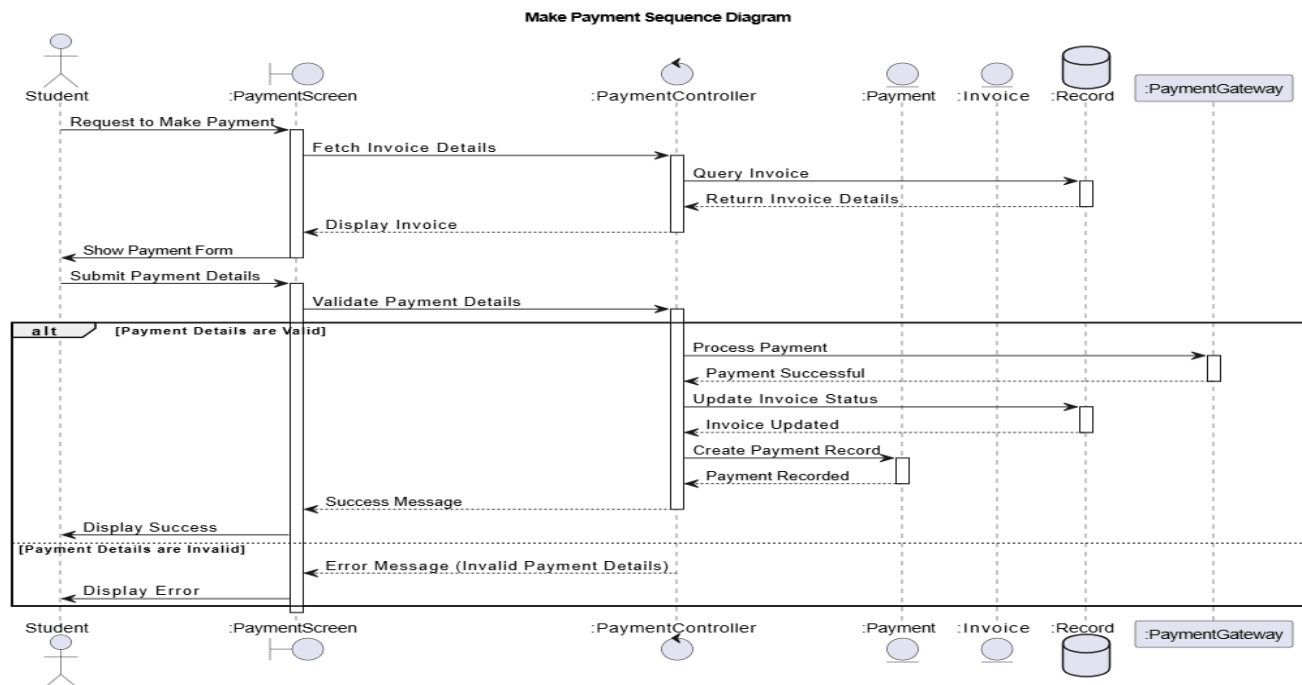


Figure VIII Student Make Payment Sequence Diagram

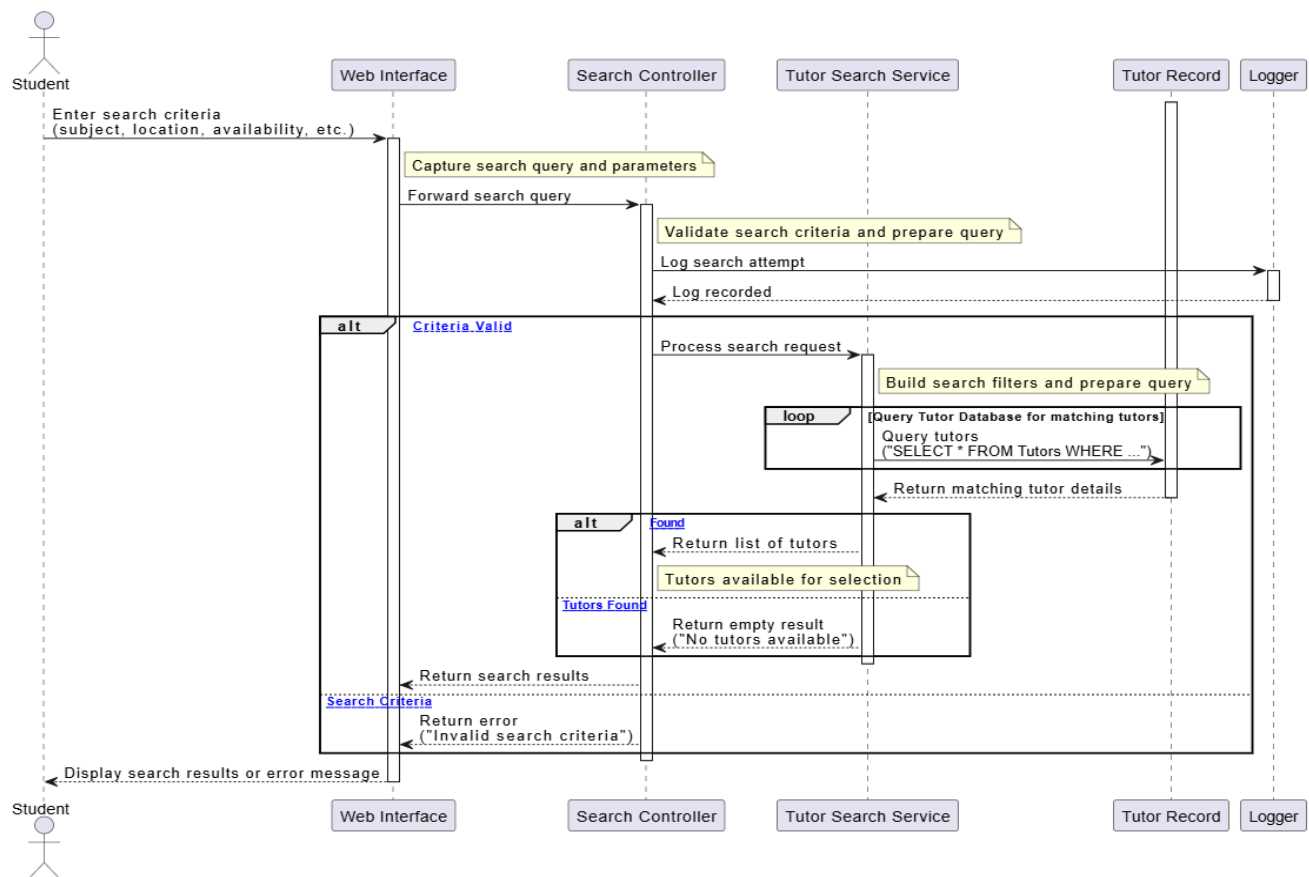


Figure IX Student Search For Tutors Sequence Diagram

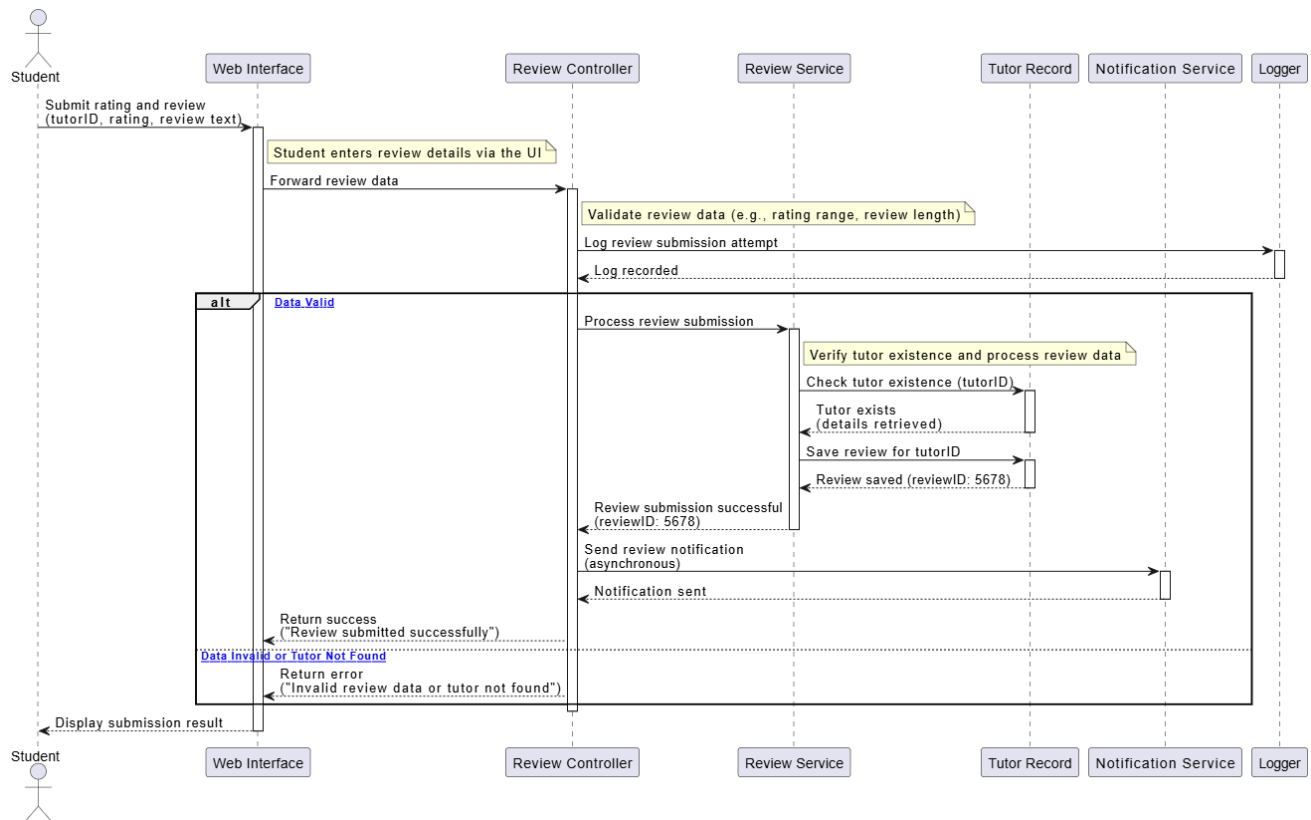


Figure X Student Rate and Review Tutor Sequence Diagram

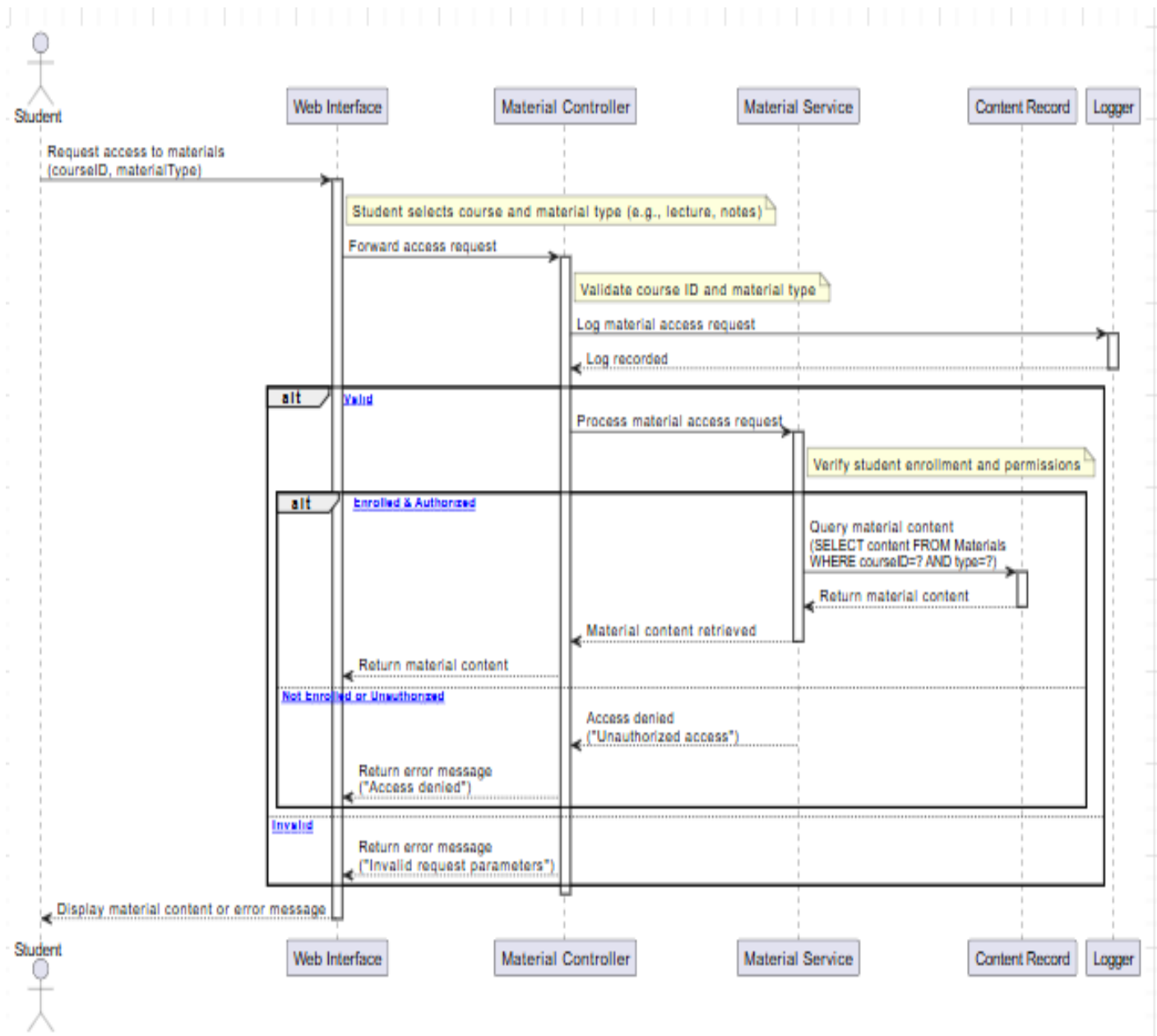


Figure XI Student Access Materials Sequence Diagram

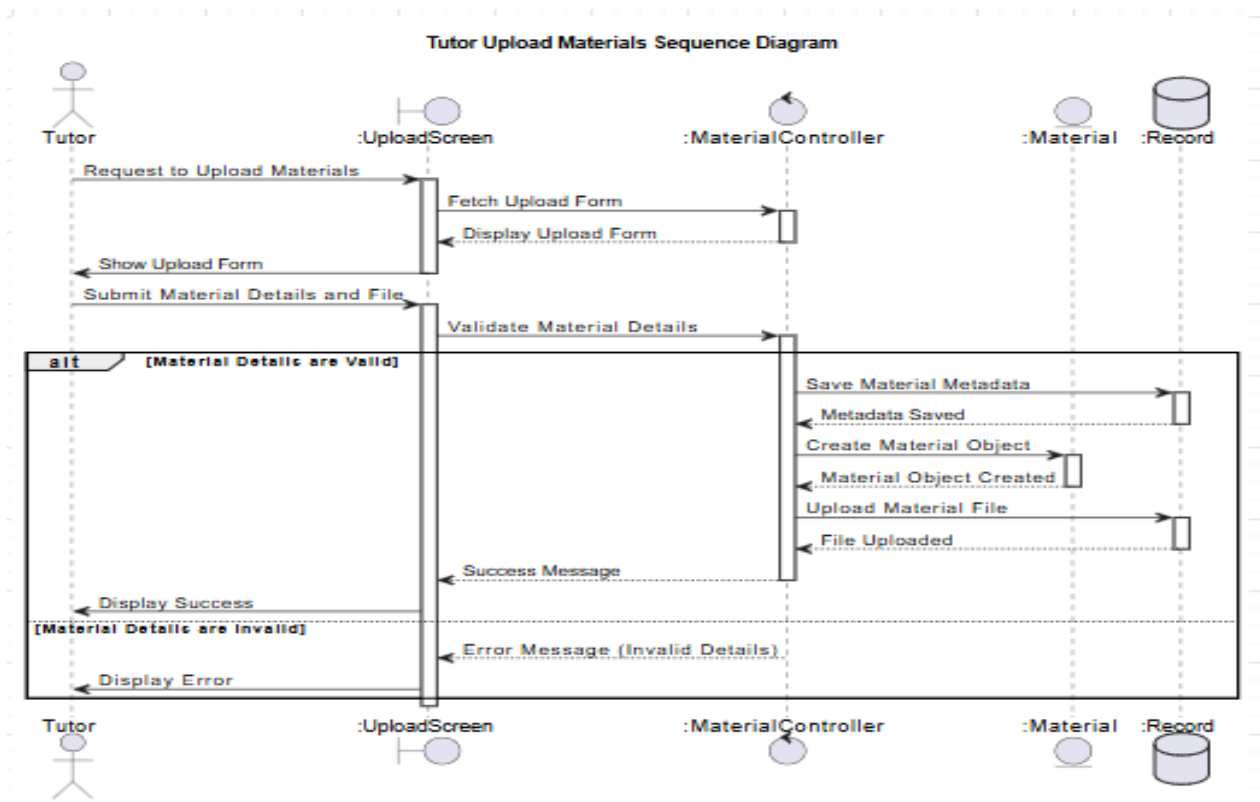


Figure XII Tutor Upload Materials Sequence Diagram

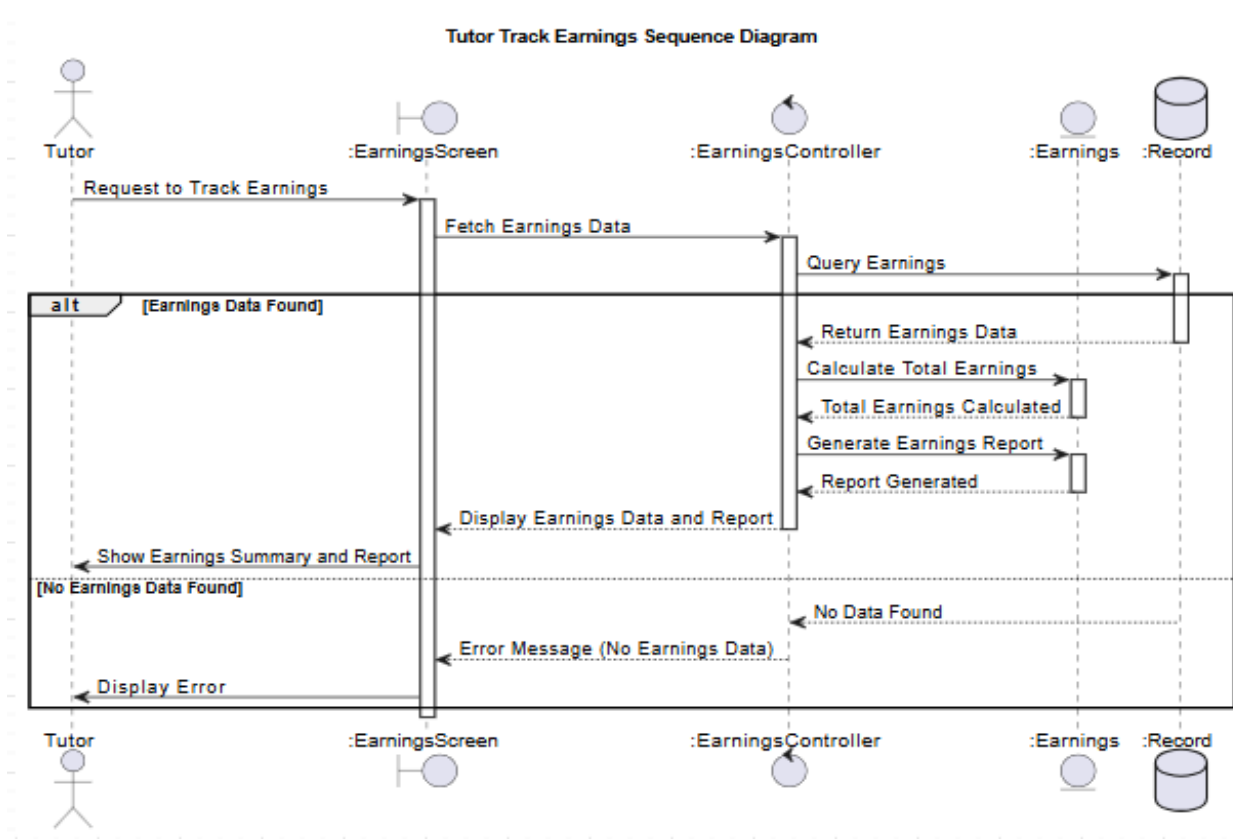


Figure XIII Tutor Track Earnings Sequence Diagram

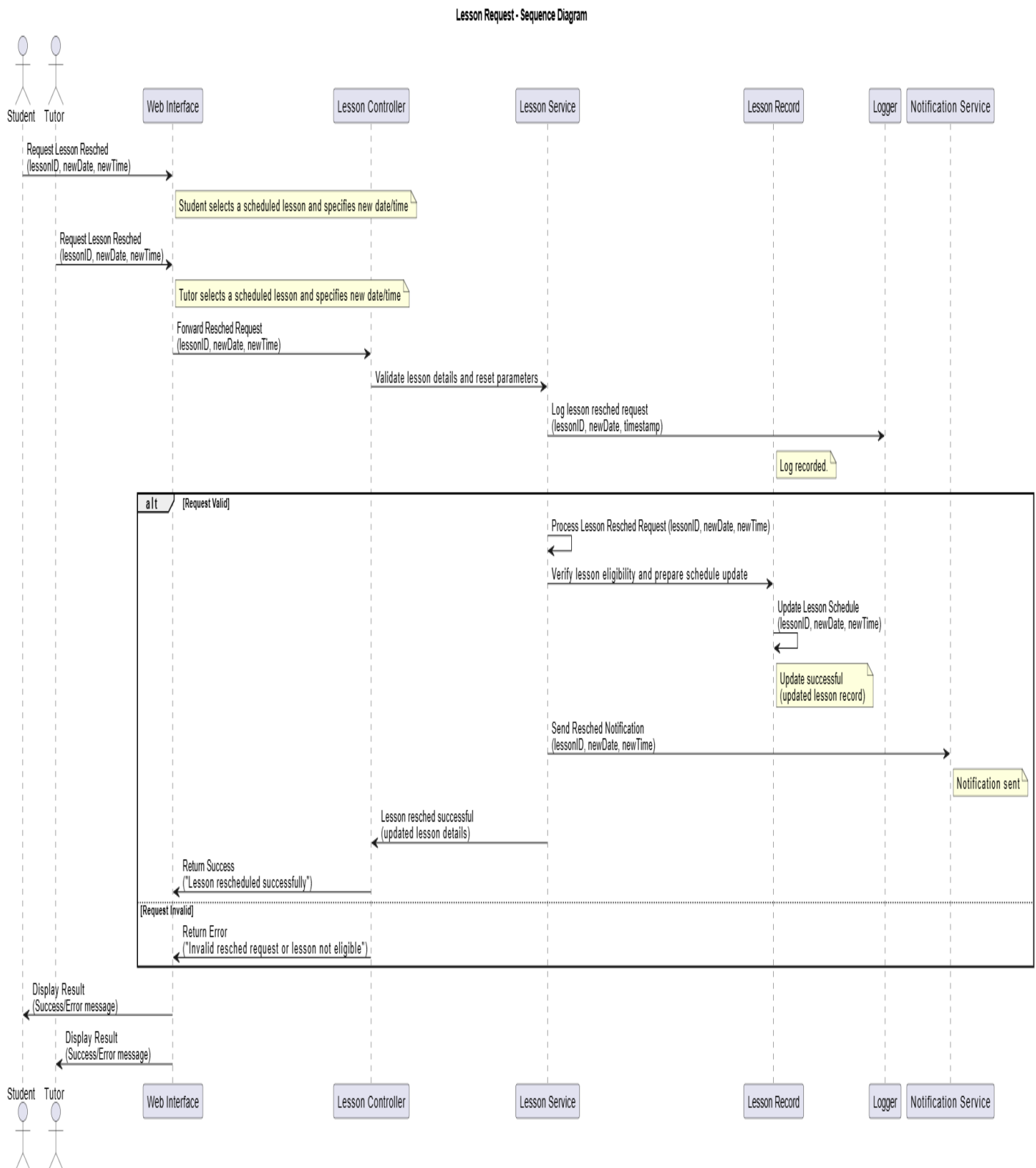


Figure XIV Lesson Reset Sequence Diagram

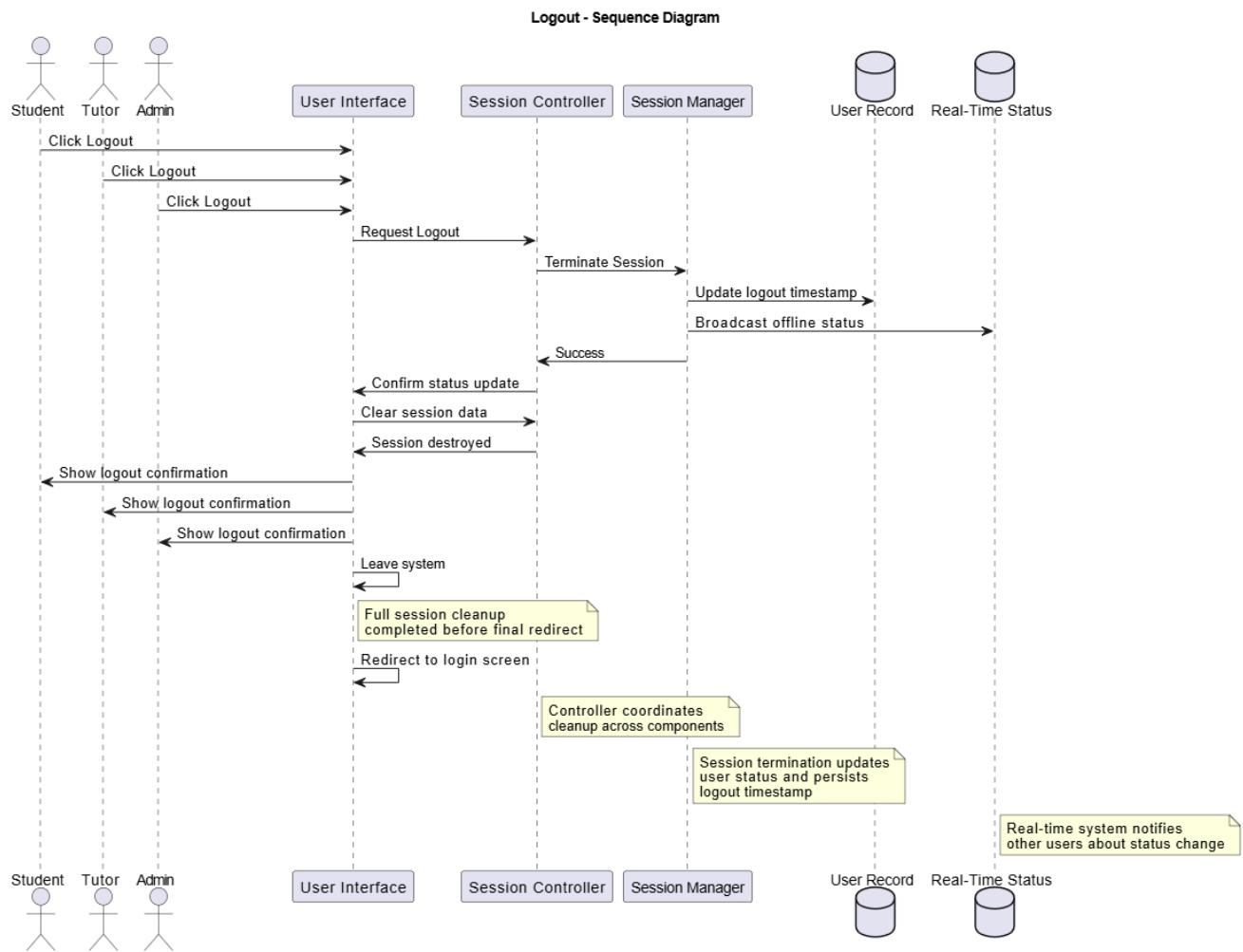


Figure XV Logout Sequence Diagram

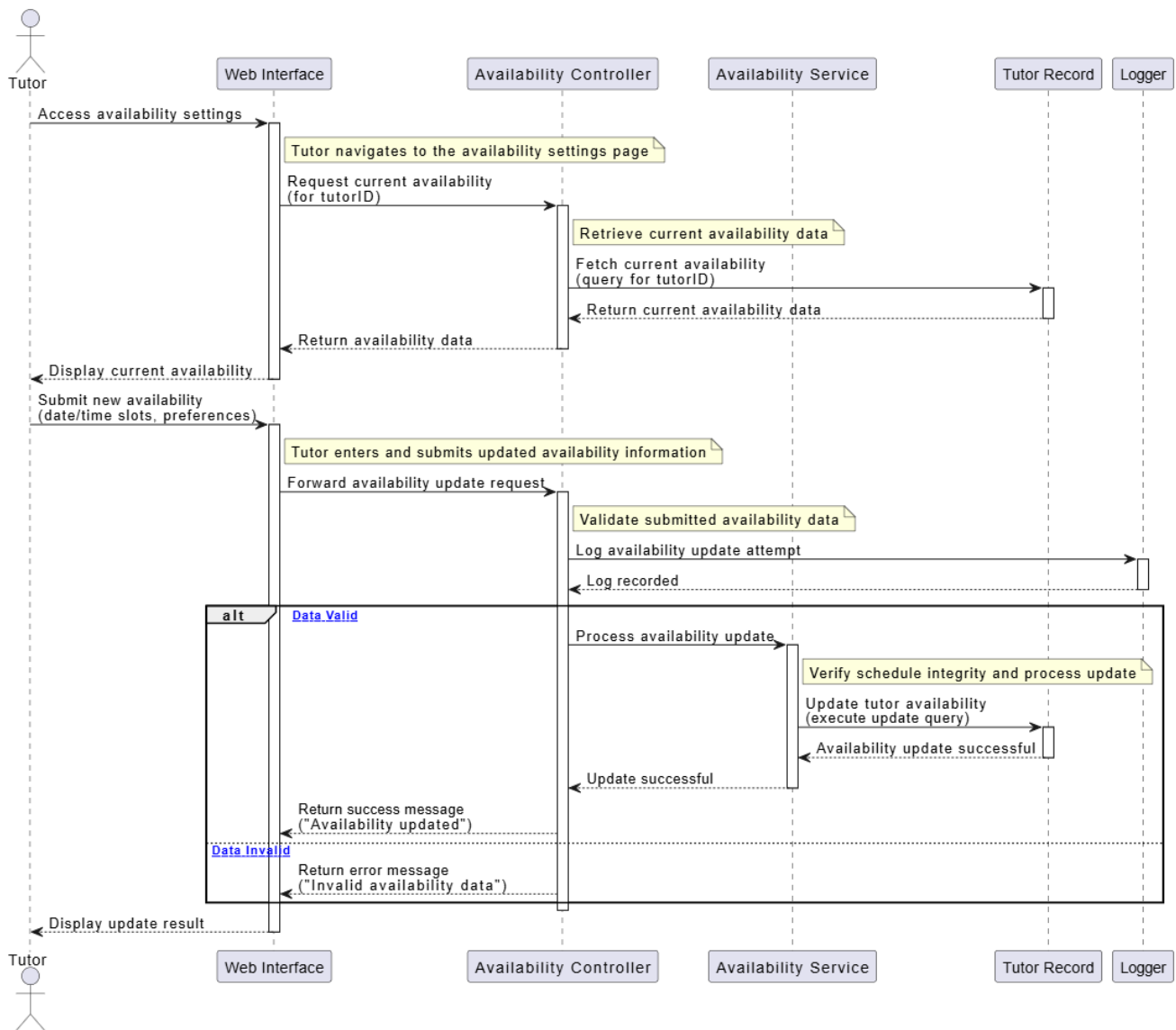


Figure XVI Tutor Set Availability Sequence Diagram

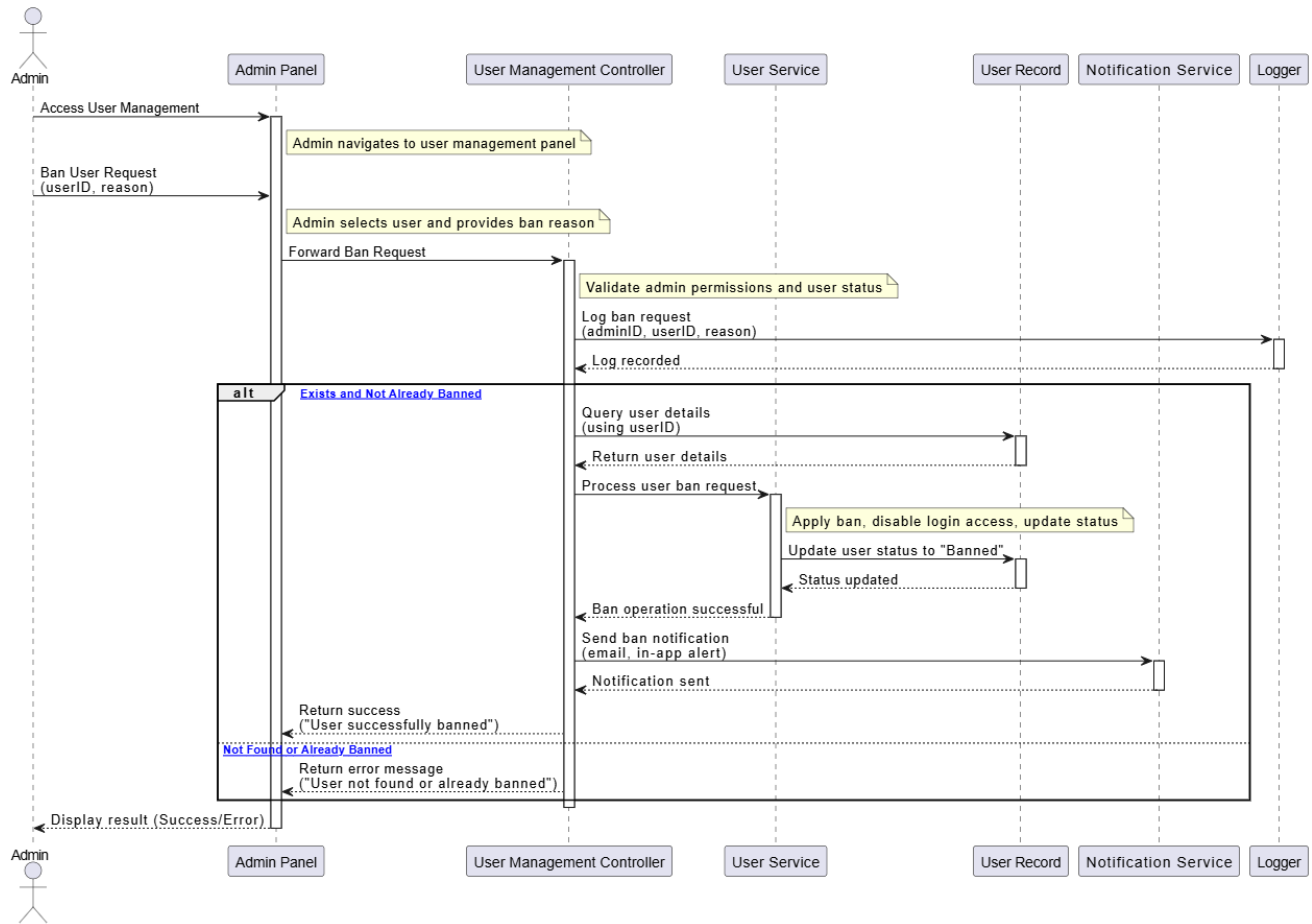


Figure XVII Admin Ban User Sequence Diagram

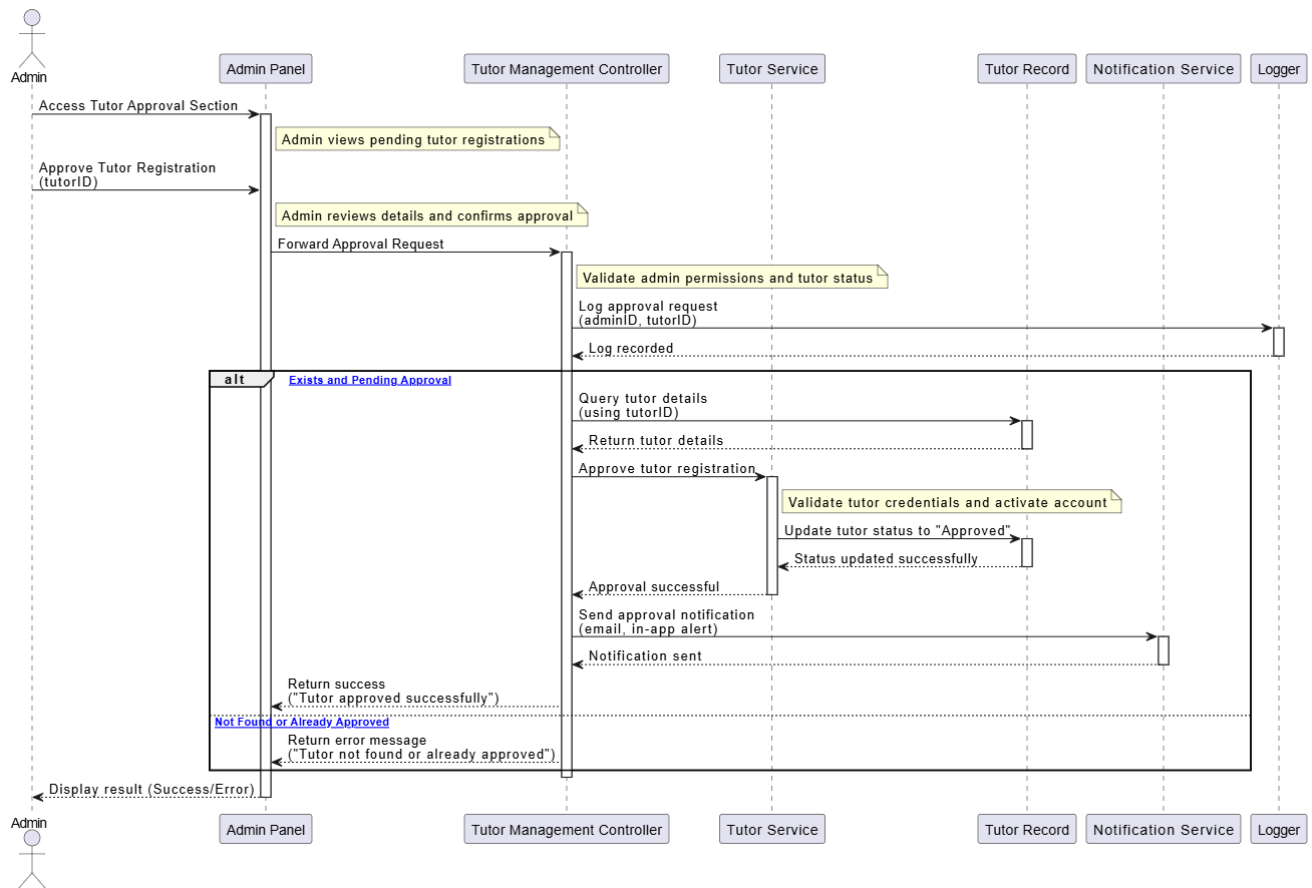


Figure XVIII Admin Approve Tutor Registration

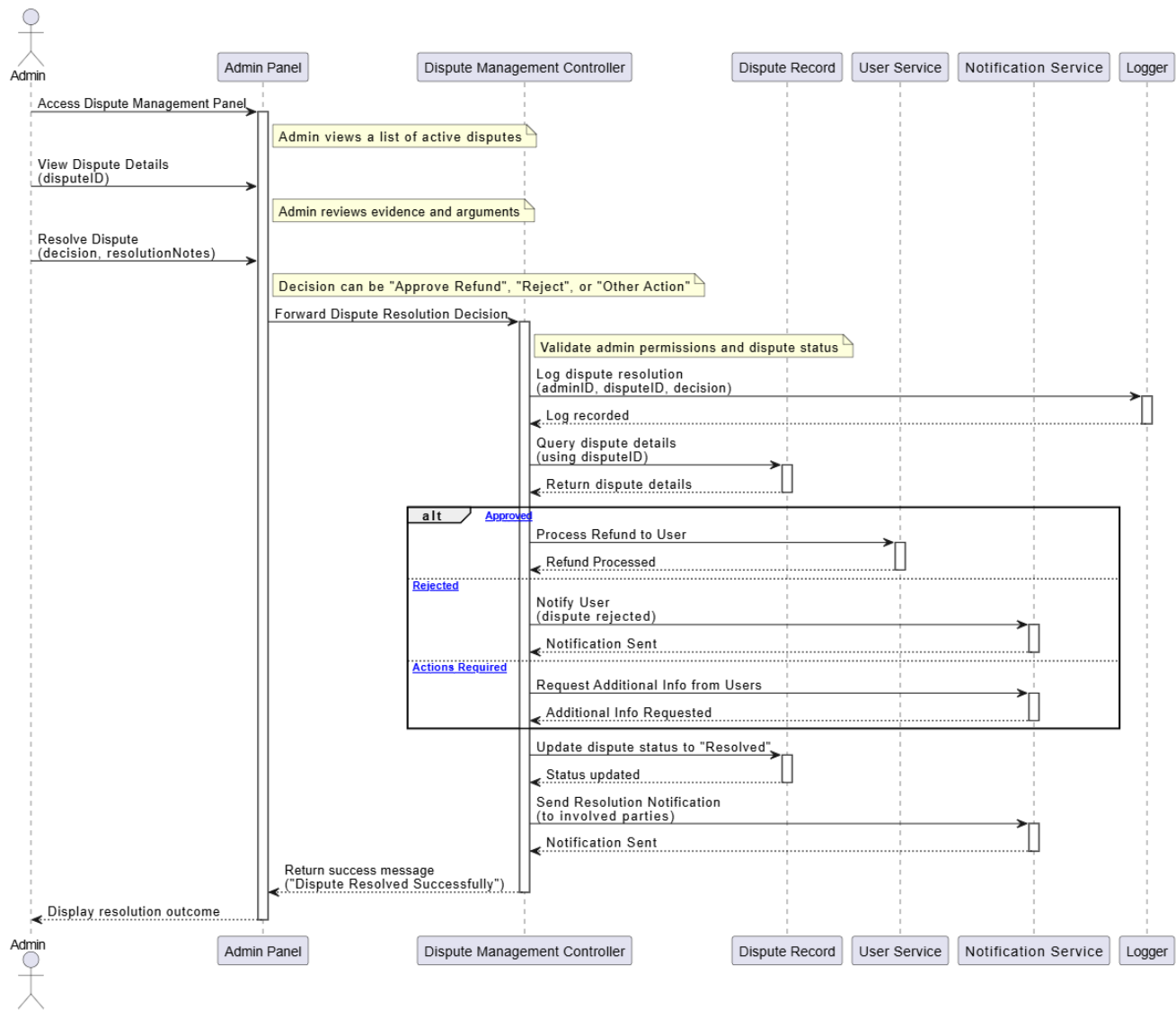


Figure XIX Admin Resolve Disputes Sequence Diagram

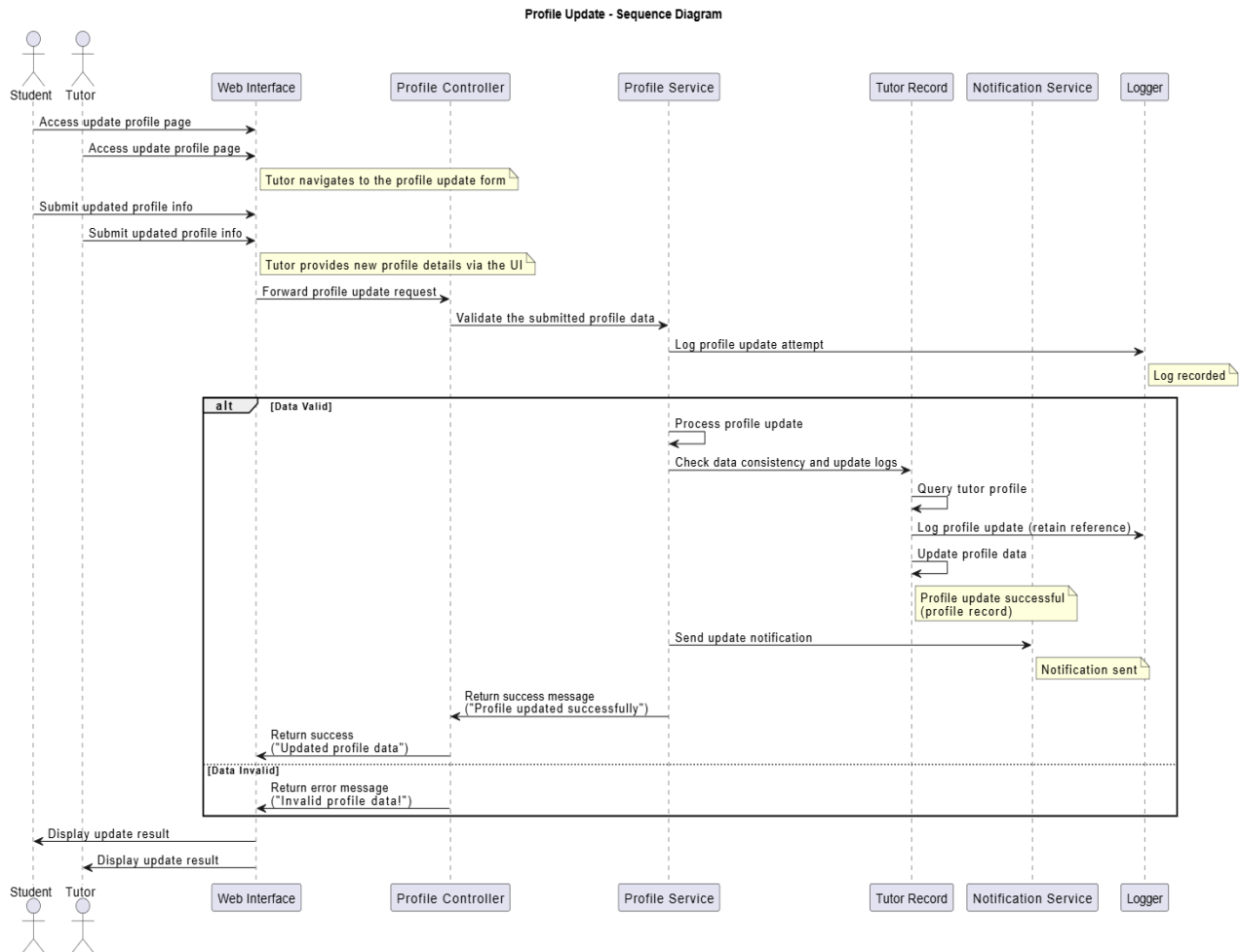


Figure XX Profile Update Sequence Diagram

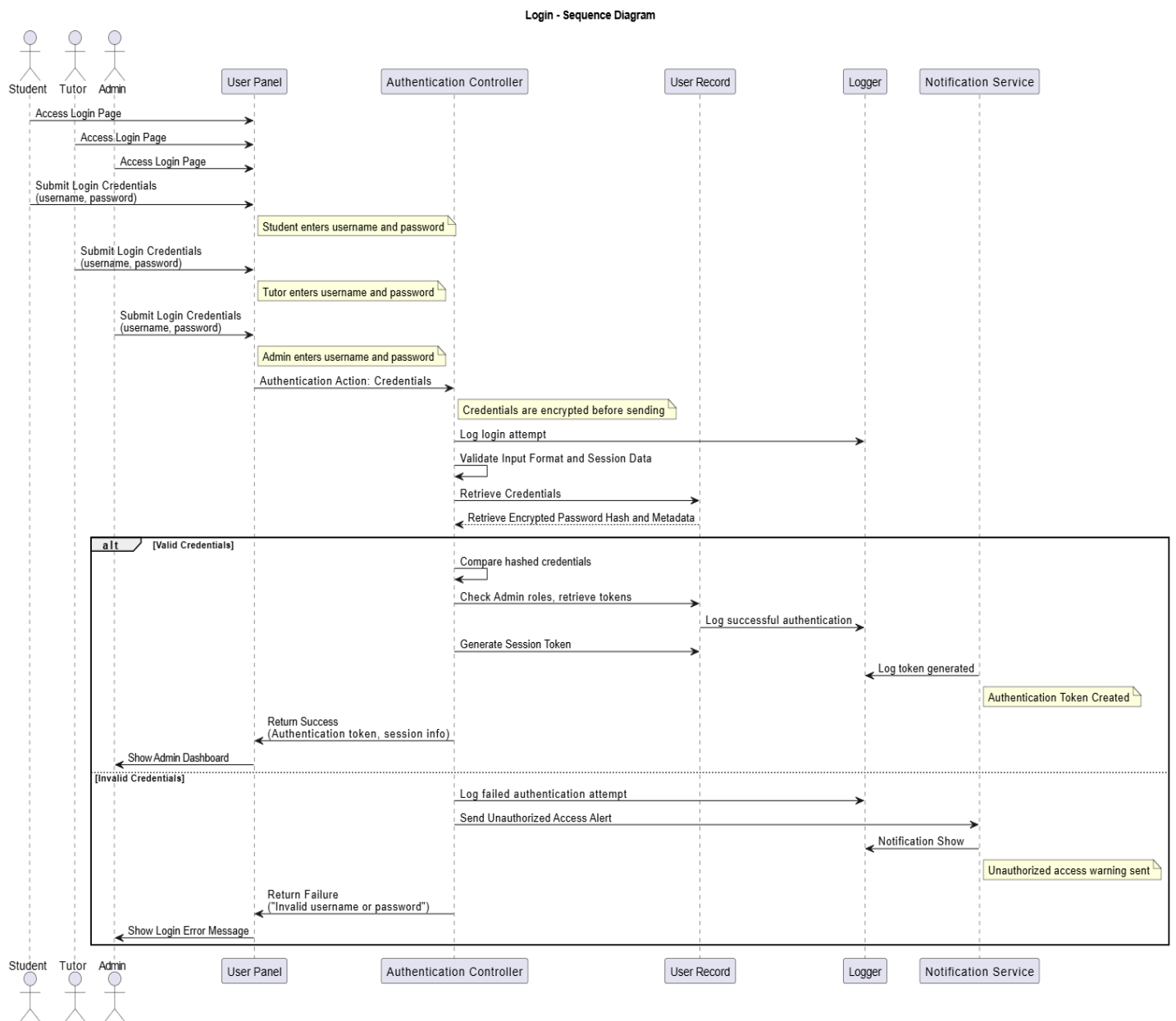


Figure XXI Login Sequence Diagram

Reset Password Sequence Diagram

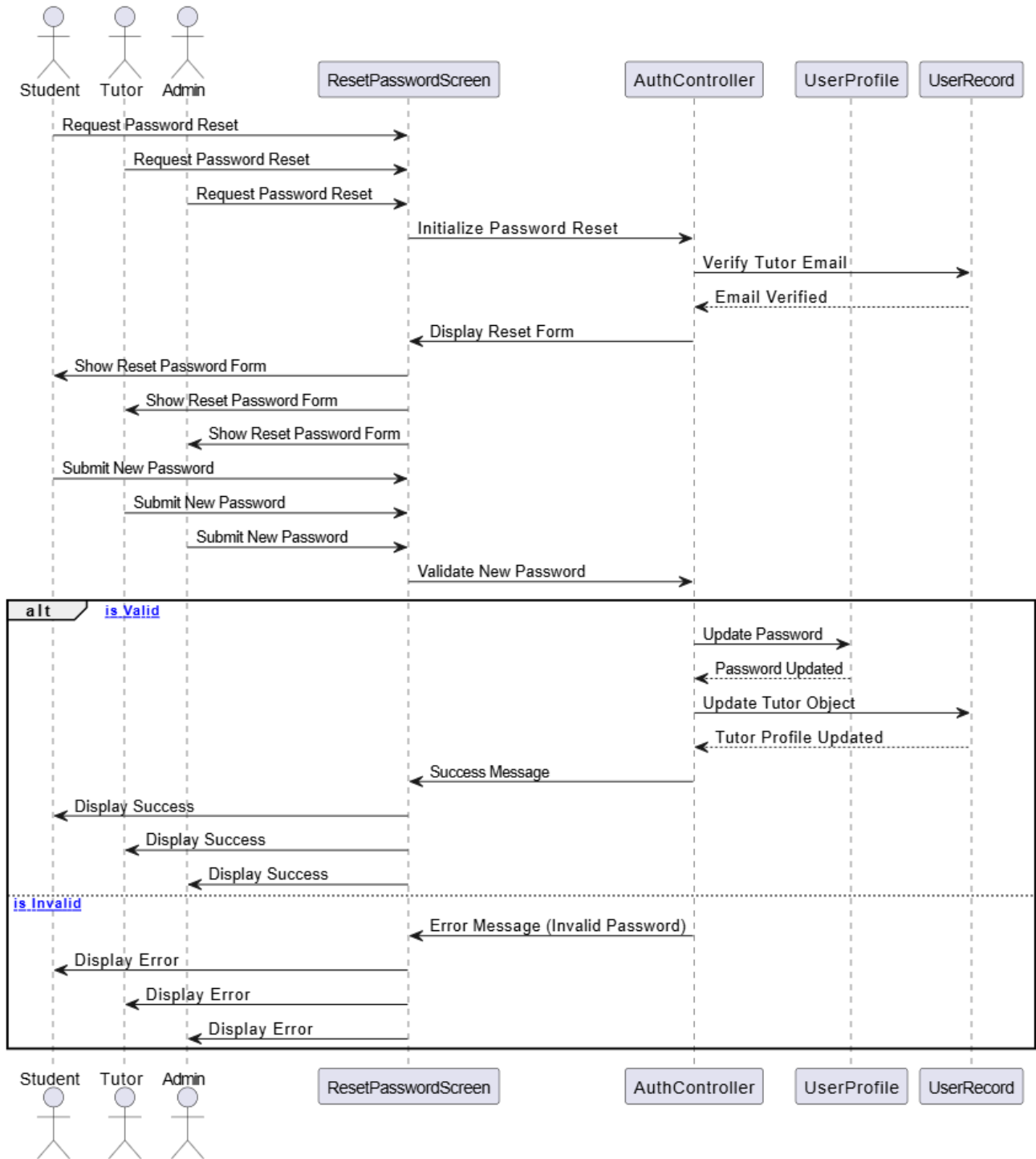


Figure XXII Reset Password Sequence Diagram

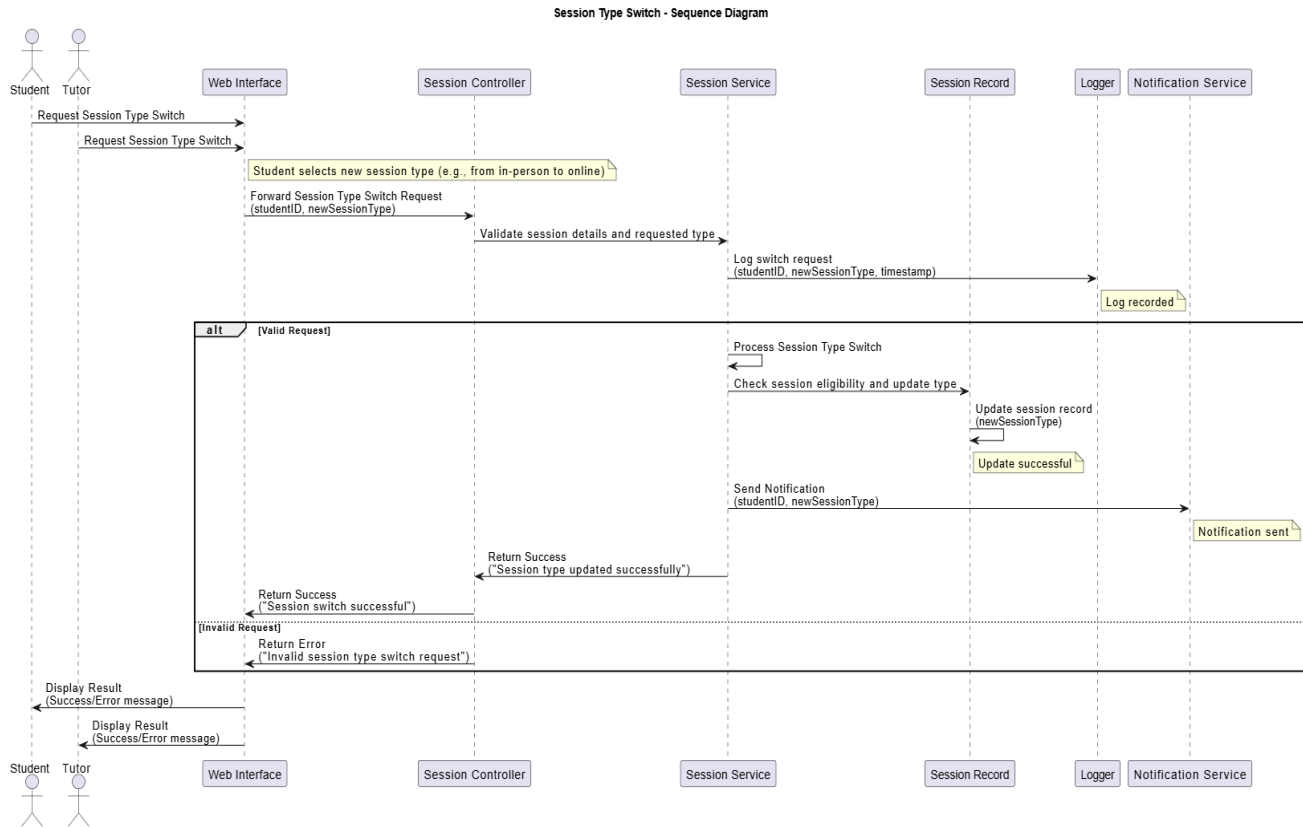


Figure XXIII Session Type Switch Sequence Diagram

2.5.3 Activity Diagram

Diagram showing the flow of activities in the system, such as:

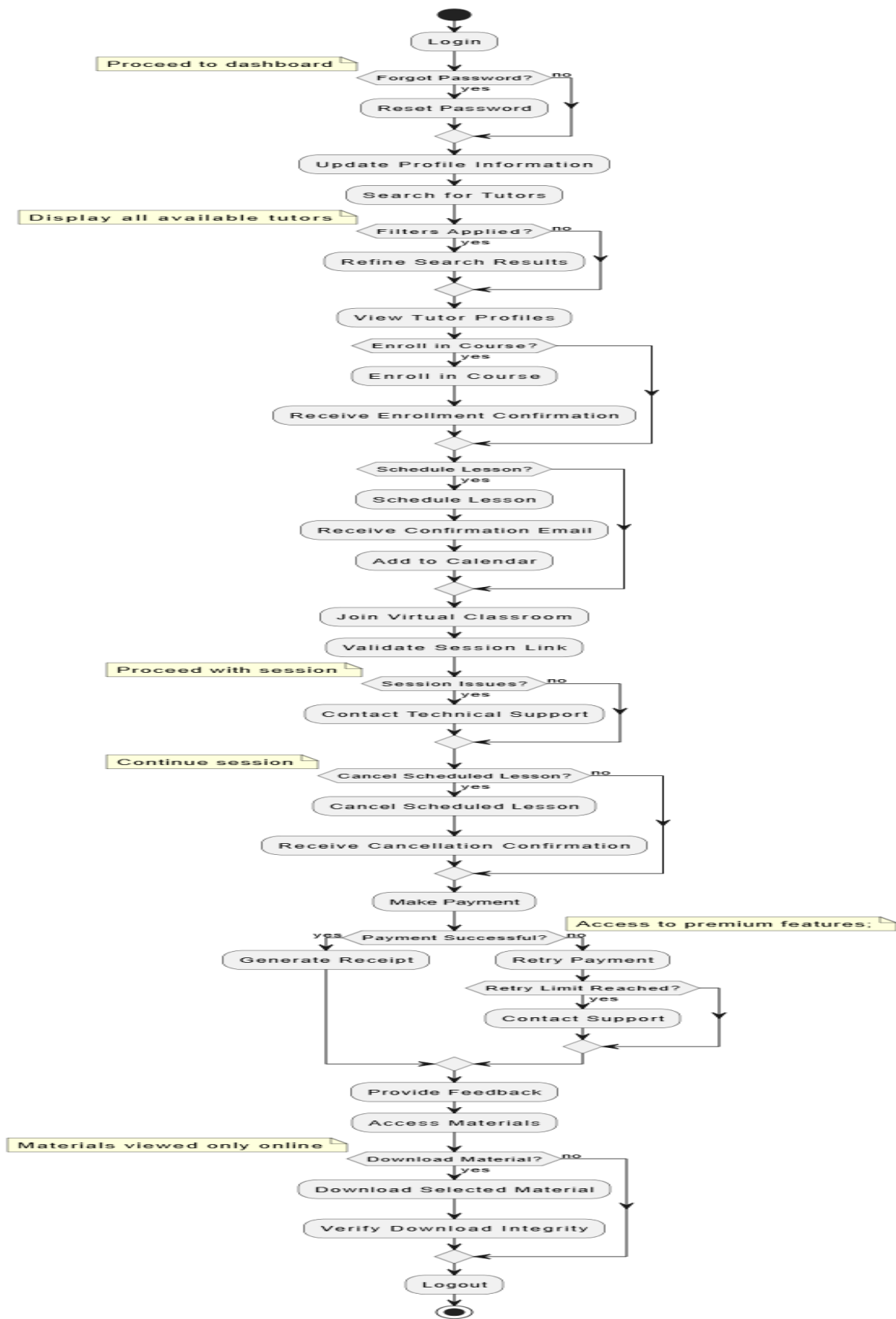


Figure XXIV student Activity diagram

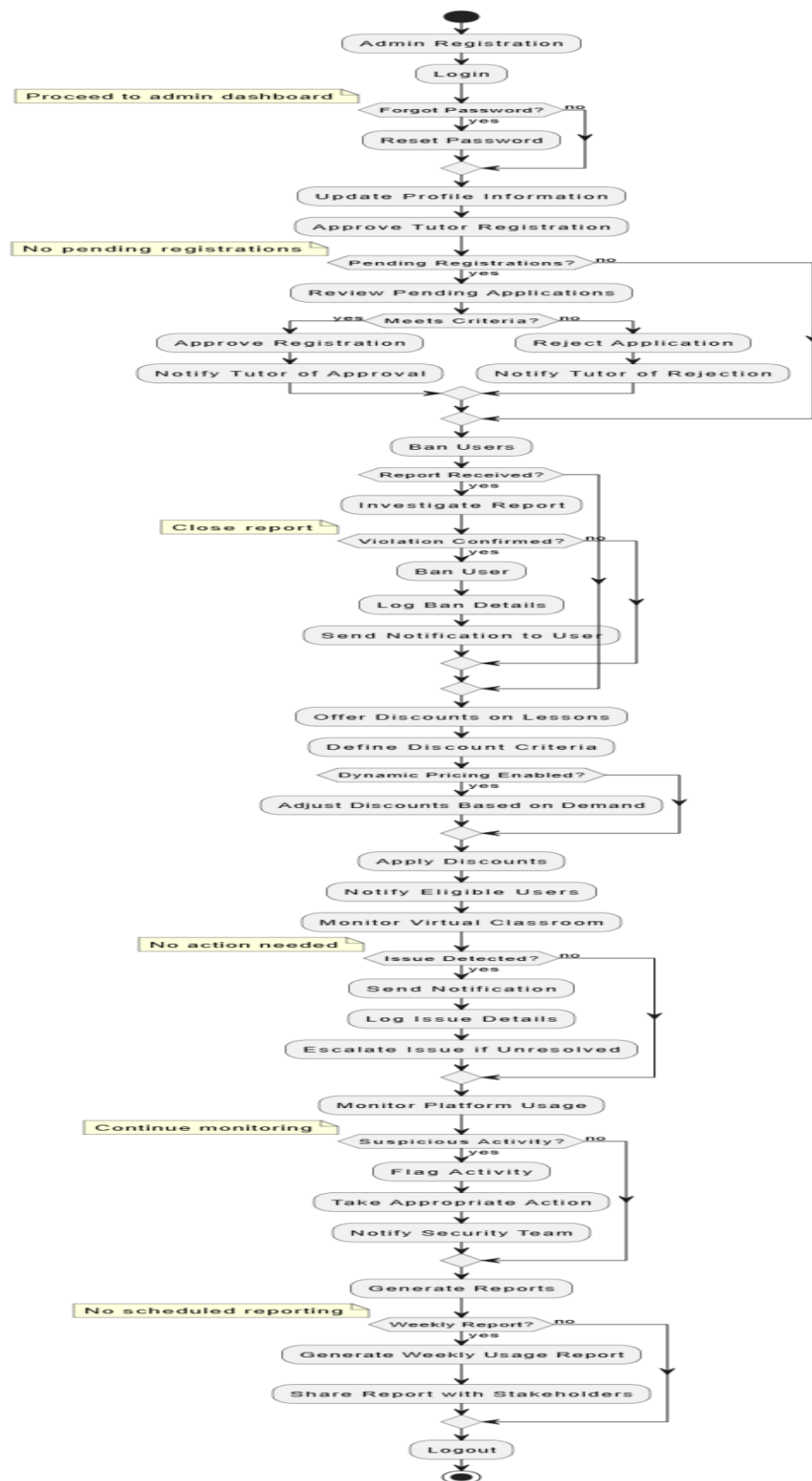


Figure XXV Admin Activity diagram



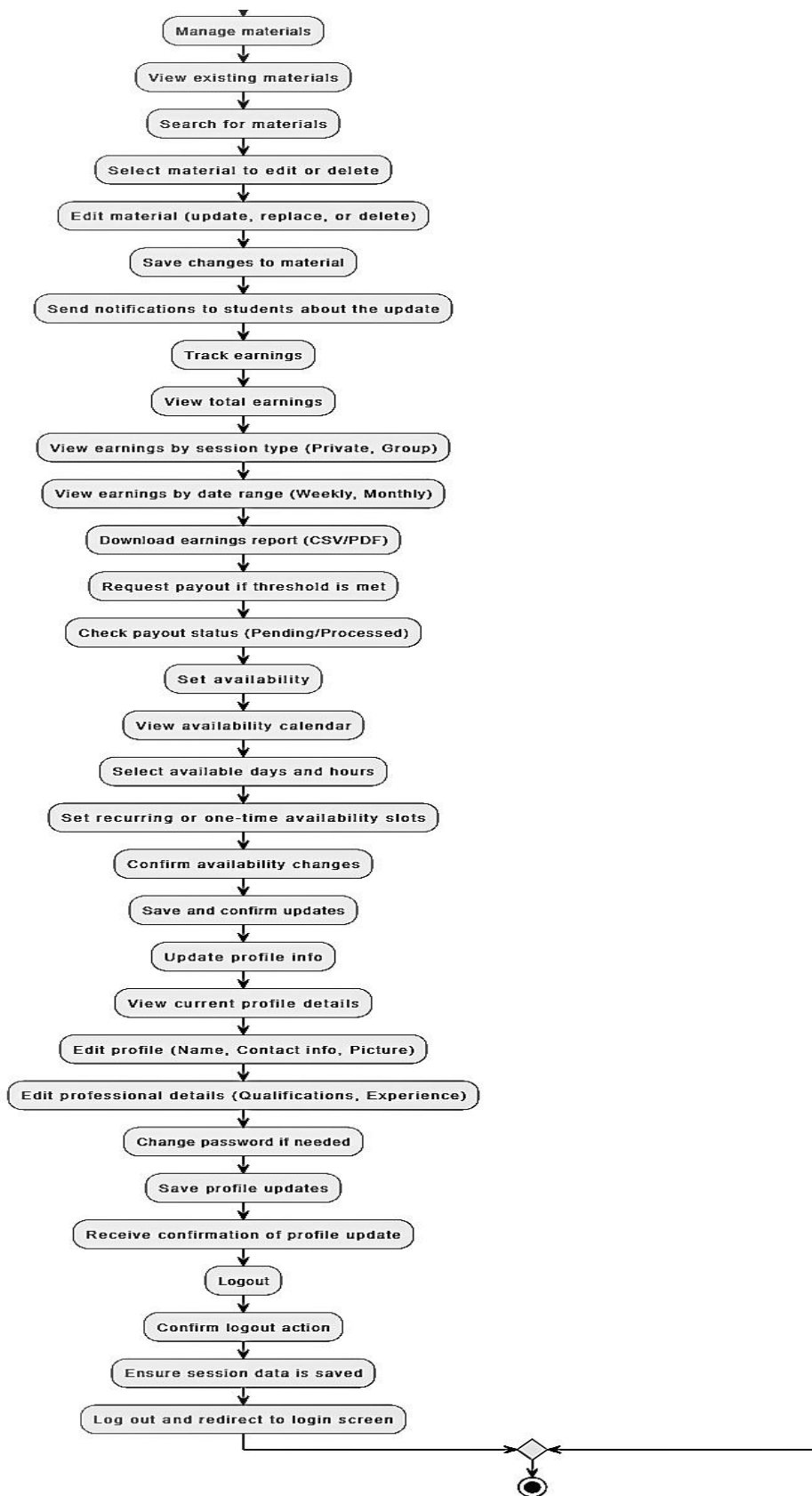


Figure XXVI Tutor activity diagram

CHAPTER THREE

SYSTEM DESIGN

3.1 Introduction

In Previous chapters the analysis model describes the system completely from the actors' point of view. In This chapter we will take the first step to the internal structure of the system architecture -- which details the system's structure, components, and their interactions, --the hardware and software mapping which aligns the software components with hardware resources for optimal performance, access control mechanism, -- which to manage user permissions and ensure secure access to different functionalities within the application-- Subsystem services -- which is a process of breaking down a larger system into smaller and more manageable components --,Persistent data management and access control --- which ensures that all critical information is stored, retrieved and maintained consistently across sessions and regulate who can view or use resources in a computing environment respectively – and class diagrams will visually represent the relationships between different components of the system, providing a clear understanding of its structure

3.2 Proposed software Architecture

3.2.1 Overview

The system is a web-based application that will be accessed through a web browser, it follows a client-server architecture and Third-party APIs for payment processing gateway and video call service, the architecture is Comprising of client layer which provide the user interface for students, tutors, and admins to interact with the system and manage their interaction. In the server side which is responsible to handle business rules and logics session management, user authentication and so on including database layer to stores and manages all persistent data, including user profiles, transaction records, session details and educational materials, ensuring data integrity and security. On the integration layer connects the application server with external APIs for payment processing, and communication purpose since they can provide faster, secure and scalable Program interface.

Client-server architecture allows for direct communication between the client interfaces and the server components, steaming data exchange without the added complexity of intermediate application layer –if three-tier architecture were to be used--. The communication between the client-side interfaces and the server side components (application server, database server) should predominantly rely on the HTTPS for enhanced security by encrypting data transmission between clients and sever.

And the communication between the client and application server will be implemented using a Representational State Transfer (REST) API. The below diagram describes the system architecture.

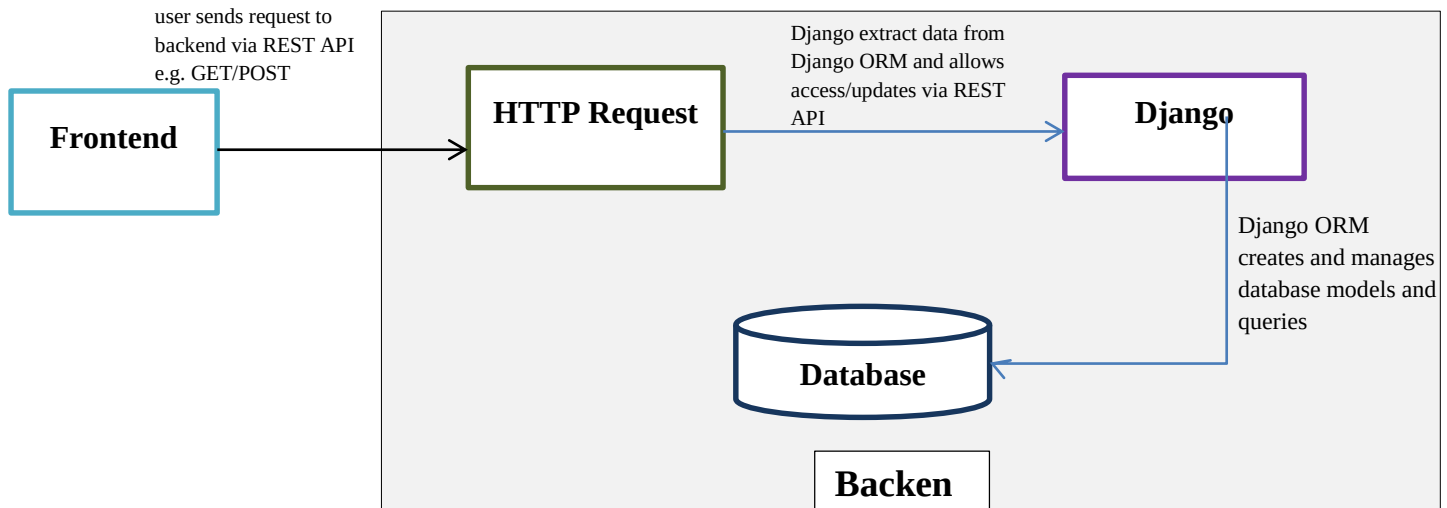


figure XXVII system architecture

3.2.2 System Decomposition

To reduce the complexity of the solution, the system is decomposed into subsystems based on functionality, which in-turn consists of a number of solution domain classes. During decomposition we have tried to achieve low coupling between subsystems so that change or error on one will not be propagated to the other and there will be a high coherence within a subsystem.

During system decomposition of this system, we break down the system into different subsystems. The subsystems are with strong coherence in order to perform specific tasks and loosely coupled in order to solve and modify the modules separately. This decomposition ensures that each subsystem is focused on a specific set of responsibilities, making the overall system modular, easier to develop, maintain, and scale. The subsystems of the Tutor Finder system are listed below

- User Management Subsystem
- Session Management Subsystem
- Availability Management subsystem
- Payment Processing Subsystem
- Chat and Messaging Subsystem
- Feedback and rating Subsystem
- Resource Management Subsystem
- Notification Management Subsystem
- Virtual Class Subsystem
- Admin Management Subsystem

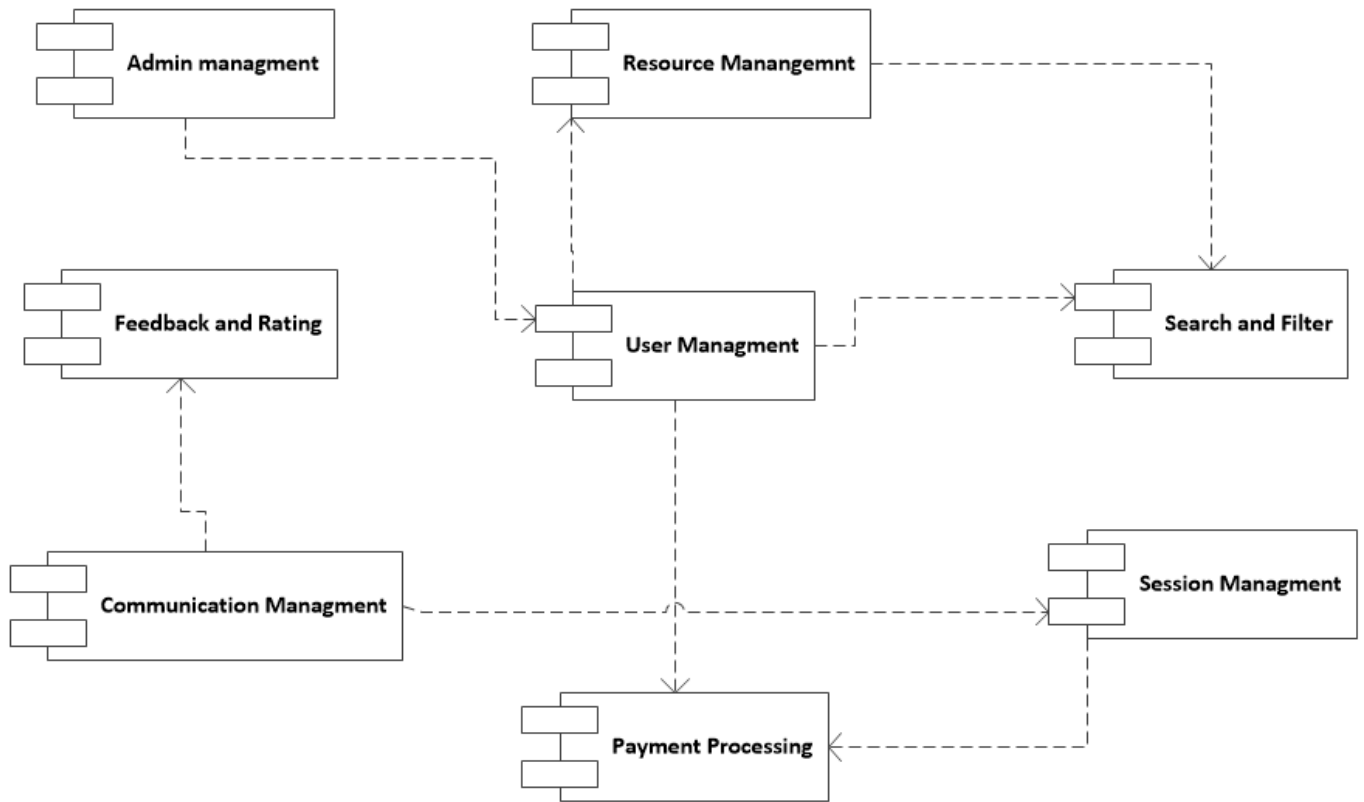


Figure XXVIII component diagram

➤ User Management Subsystem

The User Management is responsible for managing the user registration, authentication, profile update, as well as managing user roles (tutor, student, admin). And it consists of the following components.

- User Registration
- Profile management
- Authentication
- Profile Management
- Role Based Access control

➤ Session Management Subsystem

This part of the system handles the scheduling, booking and track the status and history of tutoring session. The following are components of session management system.

- Session Booking
- Calendar Integration

- Session status Management (completed, canceled, upcoming)

➤ **Payment Processing Subsystem**

Payment Processing subsystem is the one responsible for financial transactions including students payment, tutor earning, escrow and platform commissions. It includes the following components.

- Payment Gateway integration
- Escrow Management
- Commission Calculation and payout

➤ **Chat and messaging Subsystem**

The real-time communication between students and tutors is facilitated by this part of the system, this system allow the student and the tutor to have a real time conversation with maintained history of their conversation.

➤ **Notification Subsystem**

The Notification subsystem allow to send system notification and update information to the users through In-app and telegram bot to boost reachability of the notification. The notification include verification updates, payment status , schedule status and other student-tutor related updates.

➤ **Feedback and rating Subsystem**

This subsystem collect and manages feedback from students about tutors and sessions. It include the following components.

- Rating System for Tutors
- Session Feedback Forms

➤ **Resource Management Subsystem**

The purpose of this part of the system is to Allow Users (tutors) to upload and organize educational material for students.

- File Upload and Storage
- Resource Categorization and search
- Access control for shared Resources

➤ **Admin Management Subsystem**

This subsystem provides tools for admins to monitor platform activity and manage users starting from verifying a particular tutor is qualified or not for tutoring and reviewing reports and allow the admin to take action based on the business rules.

The admin management subsystem includes the following components.

- User (tutor) Verification
- System analytic
- Dispute Resolution

3.2.3 Hardware and Software Mapping

This involves mapping software components to the underlying hardware infrastructure to optimize performance and resource utilization. It ensures that the software runs efficiently on the selected hardware and Supports scalability by aligning with the system's resource needs. Since the system is a web based system, it is client-server software; the following are devices used constitute the hardware components.

- **Application Server** is where the whole backend logic runs, starting from user registration and authentication to the last feedback and rating.
- **Database Server** is a machine running database software that provides database service. The database is used to store, manage, and retrieve users file, log information and to manage other persistent data
- **Client Devices** is a machine the end-user uses to access the web, i.e. desktop, smartphone etc.

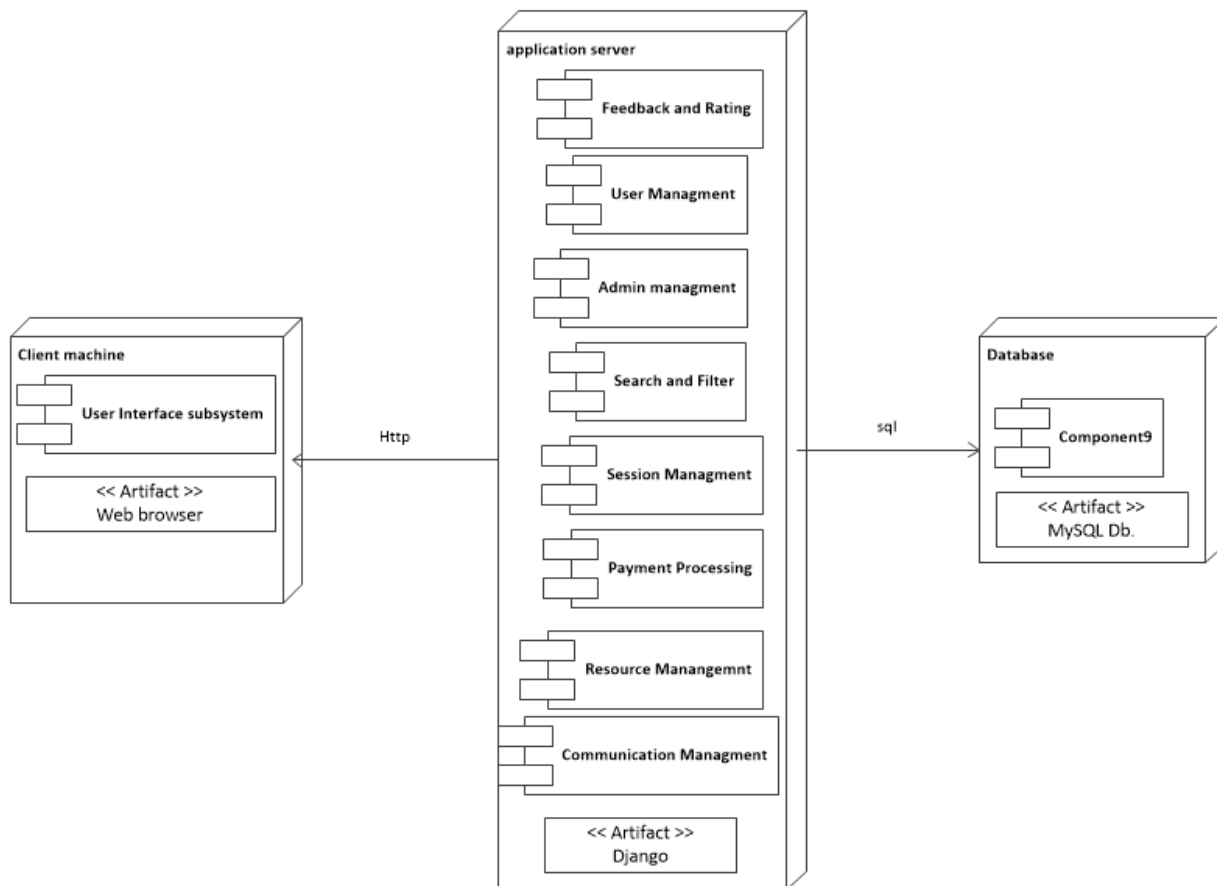


Figure XXIX Hardware to software mapping

3.2.4 Persistent Data management

Persistent data management ensures that all critical information is stored, retrieved, and maintained consistently across sessions. Persistent data management ensures that all user interactions, financial transactions, and resource uploads are reliably stored and easily retrievable, forming the backbone of the platform's functionality.

1. Data Type and Storage

- **User Data:** A relational database that includes student, tutor, and admin profiles, with attribute details like names, contact information, qualification, and preferences stored in relational tables
- **Session Data:** hold information about scheduled sessions, bookings, attendance. Each includes timestamps, session status and feedback linked to the user IDs.
- **Payment Data:** Hold transaction records, including payments status, amount, commissions, and escrow details; these ensure financial transparency and auditability.
- **Resource Data:** all uploaded educational materials, such as PDFs & notes. Files are stored in cloud storage, with metadata saved in the database.

2. Data Access and Manipulation

- **CRUD Operation:** Backend APIs handle the Create, Read, Update, and Delete Operations securely for all data types.
- **Data Relationships:** to maintain relationships between different data types -- e.g., students and tutors linked to sessions, tutors to resources -- using foreign keys
- **Indexing and optimization:** Index critical fields --e.g., user IDs, session dates—to speed up queries.

3. Security and integrity

- **Encryption:** Sensitive data like passwords and payment information is encrypted both in transit (using TLS) and at rest.
- **Data Validation:** Ensure that all inputs are validated before being stored to prevent injection attacks and maintain data integrity

3.2.5 Access Control and security

In multi systems, different actors have access to different functionality and data. During analysis, we modeled these distinctions by associating different use cases to different actors. During system design, we model access Control by examining the object model and determine which objects are shared among actors, and control those objects data and functionalities based on roles and permissions. The following table outlines the objects or their functionalities, and the access level assigned to each actor.

Table 5Access control

Objects or activities on the system	Actors		
	Student	Tutor	Administrator
Resources upload & View	View	Upload, delete	View, Delete
Notifications view & manage	View	View	Manage
Search and filters	Yes	Yes	Yes
Feedback and ratings submit, view	Submit, view	View own	View, mange
Administration monitoring and managing panel			Full Access

3.2.6 Subsystem Services

Subsystem services are the functionalities provided by each subsystem to fulfill specific requirements. We defined clear responsibilities for each subsystem to ensure that all the subsystems interact seamlessly.

- User Management Subsystem: this subsystem handles user (students, tutors) to create accounts, authentication for secure login and logout process, profile management, and assign roles.
- Search and filter subsystem: Enable users to search for tutors by subject, availability and ratings, and apply more filters to narrow down search results. Also it enable for users to search for educational material based on keywords and categories.
- Session Management Subsystem: handles the session scheduling process based on availability, and allow users to manage and modify session timings. This part of the system is also responsible of tracking the status of session (upcoming, completed, and canceled).
- Payment Management Subsystem: this subpart of the system manages all financial transactions process from students for booked sessions. Manage escrow and hold payments from students until sessions are successfully completed, issue refunds for disputed sessions. And commission calculation to deduct from tutors earnings.
- Communication Management Subsystem: To facilitate real-time communication between students and tutors this subsystem handles the text and video communication between students and tutors, and maintains a history of all communications for reference. Also it is responsible for sending Notifications and reminders for upcoming sessions and payment confirmation.

- **Feedback and Rating Subsystem:** This part of system collects and manages feedback from students about tutors and sessions. It Allow students to view and provide feedback after session and rate tutors based on session experience.
- **Resource Management Subsystem:** Allow tutors to upload and delete educational resources. While it categorize and manage uploaded resources.to help students find and view shared material more easily
- **Admin Management Subsystem:** provide administrative tools to monitor and manage the platform. Such as tracking user activities and system usage, handle disputes, verify tutors upon registration, generate reports on platform performance, user engagement and financials

3.2.7 Detail Class Diagram

A class diagram is used to show the existence of classes and their relationships in the logical view of a system. During design, we use class diagrams to capture the structure of the classes that form the system's architecture. The class diagram will emphasize the two essential elements of a class: classes and their basic relationships. The selected classes cover essential functionalities related to user management, session handling, payment processing resource allocation, communication, notifications and personalized recommendations to address the system's functionalities and meet the user's needs.

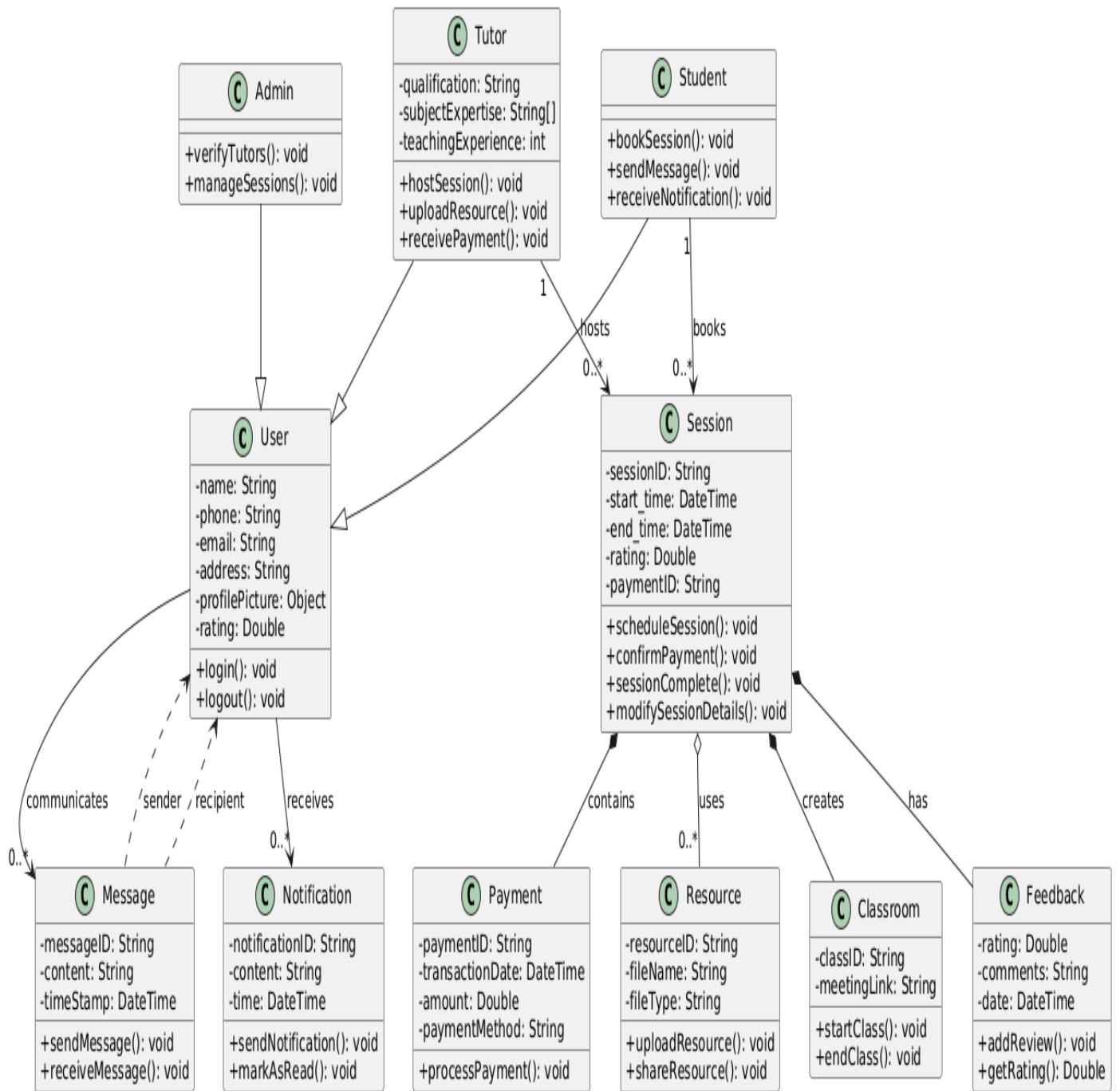


Figure XXX class diagram

3.2.8 Package

Package in object-oriented programming serves as a container to organize and group related classes, interfaces, and resources. It helps in structuring the code base, enhancing modularity, and managing dependencies within the software system. We organized the list of classes within packages based on their functionalities and relationships for a neat code structure, better maintenance, and enhance code readability.

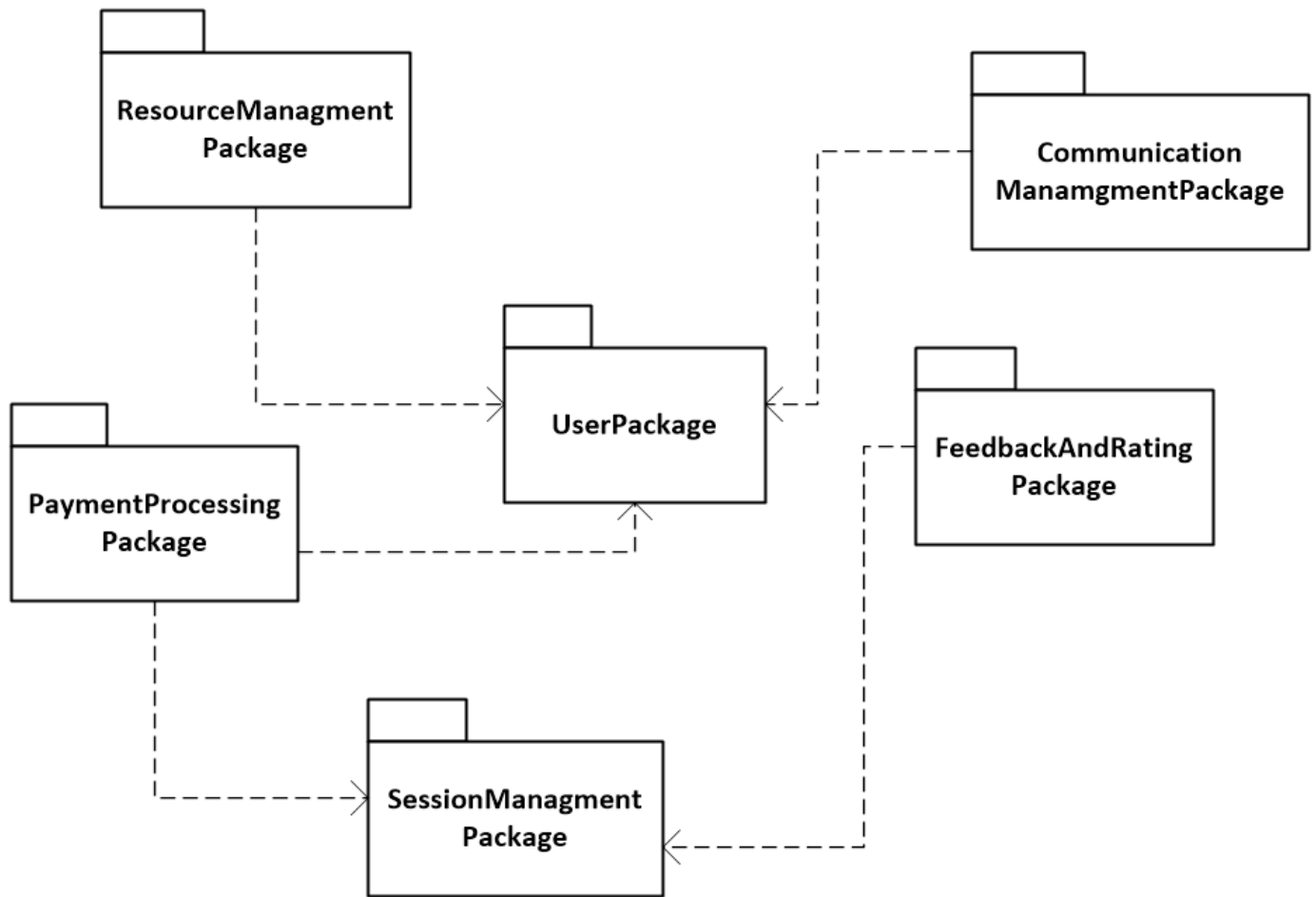


Figure XXXI package diagram

CHAPTER FOUR

IMPLEMENTATION

4.1 Introduction

The system was developed using modern **full-stack web development architecture**. The backend logic was implemented using the **Django web framework**, which provides secure authentication, database abstraction, and scalable architecture. The frontend interface was built using **HTML, CSS, JavaScript**, and Django templates to provide an interactive user experience.

The system was implemented using a **modular architecture**, where each subsystem was developed as a separate Django application. This modular approach improves maintainability, scalability, and ease of development.

4.2 Algorithms

Several algorithms and logical workflows were implemented to support the major functionalities of the system.

4.2.1 Tutor Search Algorithm

The tutor search functionality allows students to find tutors based on subject, availability, and tutor profile details.

Steps of the algorithm

- The student enters a search query or selects a subject.
- The system retrieves tutor records from the database.
- Tutors are filtered based on:
 - subject expertise
 - verification status
 - tutor availability
- The filtered tutors are displayed to the student.
 - The student may view tutor profiles and select a tutor.

4.2.2 Session Scheduling Algorithm

This algorithm manages the process of booking a tutoring session.

Steps

- Student selects a tutor.

- Student selects a subject and package.
- System checks tutor availability.
- If slot is available:
 - Create session record
 - Assign tutor and student
- Store session information in database.
- Notify tutor and student.

4.3 Description of Coding Tools

Several software tools and technologies were used during the development of the system.

4.3.1 Programming Language

The primary programming language used in this project is **Python**. Python was chosen because of its simplicity, readability, and strong support for web development frameworks.

4.3.2 Django Framework

The system backend was developed using the **Django Web Framework**.

Django provides many built-in features including:

- Authentication and authorization
- Database ORM (Object Relational Mapping)
- Secure form handling
- URL routing
- Admin dashboard

These features significantly reduced development time and improved system security.

4.3.3 Frontend tools

The user interface was developed using the following tools:

- **HTML** for page structure
- **CSS** for styling and layout
- **JavaScript** for interactivity
- **HTMX** for Dynamic behavior

These technologies allowed the system to provide a clean and user-friendly interface across different devices.

4.3.4 Database System

The system uses **SQLite** as the primary database during development. SQLite was selected because it is lightweight and Easy to configure. It can be Integrated directly with Django. All application data such as users, tutors, sessions, payments, and resources are stored in the database.

4.3.5 Version Control

The system source code is maintained using **Git version control** and hosted on **GitHub**. This enables collaborative development, version tracking, and backup of project files.

4.3.6 Development Environment

The development environment used for this project includes:

- **Visual Studio Code** – code editor
- **Python Virtual Environment** – dependency isolation
- **GitHub** – source code hosting

4.4 Code for Major Functionalities

This section describes the implementation of key system functionalities.

4.4.1 User Registration and Authentication

The system allows users to register as either a student or a tutor. Django's built-in authentication system manages user login, logout, and session management.

```
//Student Registration and Authentication Code Snippet

def student_register(request):
    if request.user.is_authenticated:
        if request.user.role == 'tutor':
            return redirect('tutor_dashbord')
        else:
            return redirect('student_dashboard')

    if request.method=='POST':
        form = StudentRegisterForm(request.POST)
        if form.is_valid():
            user = form.save(commit=False)
            user.role = 'student'
            user.save()

            StudentProfile.objects.create(
                user = user,
                favorite = 'something'
            )

            return redirect ('student_dashboard')
        else:
            form = StudentRegisterForm()
            return render(request, 'auth/student_reg.html', {'form': form})
def login_view(request):
    if request.user.is_authenticated:
        if request.user.role == 'tutor':
            return redirect('tutor_dashbord')
```

```

        else:
            return redirect('student_dashboard')
    if request.method=='POST':
        form = loginForm(request.POST, data=request.POST)
        username = request.POST.get('username')
        pwd = request.POST.get('password')

        user = authenticate(request, username=username, password=pwd)
        if user is not None:
            login(request, user)
            if request.user.role == 'tutor':
                return redirect('tutor_dashbord')
            else:
                return redirect('student_dashboard')
    else:
        form =loginForm()
        context = {'form': form}
        return render(request, 'auth/auth.html', {'form': form})

```

4.4.2 Profile Management

Tutors can manage their professional profiles within the system.

```

@login_required(login_url='/user/login/')
def my_profile(request):
    user = request.user
    avatar_form = AvatarForm(request.POST,request.FILES,instance=request.user)
    user_form = UserProfileEditForm(request.POST or None, instance=user)
    tutor_form = None
    student_form = None

    if user.role == 'tutor':
        bank_list = ChapaPayment().bank_list()
        tutor_data = get_object_or_404(TutorProfile, user=user)
        tutor_form = TutorProfileEditForm(request.POST or None,request.FILES or
None, instance=tutor_data)
        ctx = {'tutor':tutor_data, 'bank_list':bank_list}
    elif user.role == 'student':
        student_data = get_object_or_404(StudentProfile, user=user)
        student_form = StudentProfileEditForm(request.POST or None,
instance=student_data)
        ctx = {'student':student_data}

    if request.method == 'POST':
        if user_form.is_valid():

```

```

        user_form.save()

        if tutor_form and tutor_form.is_valid():
            tutor_form.save()

        if student_form and student_form.is_valid():
            student_form.save()
    if avatar_form.is_valid():
        avatar_form.save()
        return redirect('my_profile')

context = {
    'user_form': user_form,
    'tutor_form': tutor_form,
    'student_form': student_form,
    'avatar_form': avatar_form
}
context.update(ctx)
return render(request, 'sessions/profile_settings.html', context)

```

4.4.3 Availability Management

The system includes a Availability management module that allows students and tutors to manage tutoring schedule.

```

def set_availability(request):
    try:
        avail_obj = Availability.objects.filter(tutor=request.user).first()
        availability = avail_obj.availability if avail_obj else []
    except Exception:
        availability = []

    packages = TutorPackage.objects.filter(tutor=request.user)

    if request.method == 'POST':
        json_data = request.POST.get('availability_json')
        try:
            with transaction.atomic():

                data = json.loads(json_data)
                Availability.objects.update_or_create(
                    tutor=request.user,
                    defaults={'availability': data}
                )
            return redirect('set_availability')
        except Exception as e:
            return redirect('set_availability')

```

```

ctx= get_tutor_dashboard_context(request.user)
context = {
    'availablity__json': json.dumps(availablity),
    'packages': packages
}
context.update(ctx)
return render(request, 'tutor/pages/set_availablity.html', context)

```

4.4.4 Virtual Classroom

The platform integrates a **virtual classroom system** using the Jitsi video conferencing platform. Allowing tutors and students to conduct online sessions directly through the system.

```

@login_required
def join_virtual_class(request, session_id):
    """Allow tutor or student to join the virtual classroom."""
    session = get_object_or_404(BookedSession, id=session_id)

    if request.user not in [session.base_session.tutor, session.base_session.student]:
        messages.error(request, "You are not enrolled in this class.")
        return redirect("session_list")

    if not session.room_name:
        messages.error(request, "This class does not have an active virtual room yet.")
        return redirect("session_list")

    jitsi_config = {
        'domain': getattr(settings, 'JITSI_DOMAIN', 'meet.jit.si'),
        'room_name': session.room_name,
        'userInfo': {'displayName': request.user.username},
        'configOverwrite': {
            'startWithAudioMuted': True,
            'startWithVideoMuted': False,
            'prejoinPageEnabled': getattr(settings, 'JITSI_PREJOIN_ENABLED', False),
        }
    }

    return render(
        request,
        "virtualclass/classRoom.html",
        {
            "session": session,
            "jitsi_config": jitsi_config,
            "whiteboard": getattr(settings, 'JITSI_WHITEBOARD_ENABLED', False),

```

```

        "username": request.user.username,
    },
)

```

4.4.7 Payment Integration

The system integrates **Chapa payment gateway** to handle online payments. Students can pay for tutoring sessions securely through the platform.

```

def initialize_transaction(self, email, amount, tx_ref, first_name, last_name,
callback_url=None):
    """Initialize payment to platform account"""
    url = "https://api.chapa.co/v1/transaction/initialize"
    payload = {
        "amount": str(amount),
        "currency": "ETB",
        "email": email,
        "first_name": first_name,
        "last_name": last_name,
        "tx_ref": tx_ref,
        "callback_url": callback_url,
        "customization": {
            "title": "Tutor Sessio",
            "description": "Payment held by platform until session completion"
        }
    }

    try:
        response = requests.post(url, headers=self.get_headers(),
data=json.dumps(payload))
        response.raise_for_status()
        return response.json()
    except requests.exceptions.RequestException as e:
        if hasattr(e, 'response') and e.response:
            return None

```

4.5 Testing

System testing was conducted to ensure that all modules function correctly and meet system requirements.

4.5.1 Unit Testing

Individual components such as models, views, and forms were tested independently to ensure correct behavior.

```

class SimpleUserTests(TestCase):

```

```

def setUp(self):
    self.User = get_user_model()

def test_create_user_and_login(self):
    """Create a user, verify role/defaults and that login works."""
    user = self.User.objects.create_user(
        username='testuser', password='password123', first_name='Test',
last_name='User'
    )

    # default role should be 'student'
    self.assertEqual(user.role, 'student')
    self.assertTrue(user.is_student())
    self.assertFalse(user.is_tutor())

    # string representation
    self.assertEqual(str(user), 'testuser')

    # Login via the test client
    logged_in = self.client.login(username='testuser', password='password123')
    self.assertTrue(logged_in)

```

4.5.2 Integration Testing

Integration testing was performed to verify that different modules work together correctly.

```

booked = BookedSession.objects.create(
    base_session=bs,
    start_time='10:00',
    end_time='11:00',
    session_type='online'
)

self.assertEqual(booked.base_session, bs)

def test_start_and_end_session_updates_remaining_hours(self):
    bs = BaseSession.objects.create(
        student=self.student,
        tutor=self.tutor,
        subject_name='Physics',
        price=Decimal('50.00'),
        total_hours=Decimal('2.00'),
        remaining_hours=Decimal('2.00'),
        remaining_sessions=2
    )

    # simulate a session that started 1 hour ago
    bs.current_start_time = timezone.now() - timedelta(hours=1)

```

```
bs.save()

# call end_session which should subtract ~1 hour from remaining_hours
bs.end_session()

# remaining_hours should be less than initial (allowing float rounding)
self.assertLess(float(bs.remaining_hours), 2.0)
# status should be either 'ongoing' or 'completed' depending on subtraction
self.assertIn(bs.status, ('ongoing', 'completed'))
```

4.5.3 System Testing

System testing was performed to evaluate the entire application as a complete system.

The following scenarios were tested:

- User registration and login
- Searching for tutors
- Scheduling tutoring sessions
- Joining virtual classes
- Uploading and downloading resources

All major system functions were verified to ensure they work correctly.

CHAPTER FIVE

CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

The **Home Tutor Finder System** was developed to address the challenges students face when trying to find qualified tutors and schedule tutoring sessions efficiently. The system provides a centralized platform where students can discover tutors, book sessions, communicate with instructors, and access learning resources.

This chapter summarizes the outcomes of the project and evaluates how effectively the system fulfills its intended objectives.

5.1 Recommendations

Several improvements can be implemented in future versions of the system to enhance its capabilities.

- **AI-Based Tutor Recommendation**

Machine learning algorithms could be used to recommend tutors based on:

- student learning preferences
- previous sessions
- tutor ratings

This would improve the efficiency of tutor discovery.

- **Mobile Application Development**

Developing a **mobile application** would allow students and tutors to access the platform more conveniently using smartphones.

- **Improved Security Features**

Future improvements could include:

- enhanced authentication methods
- stronger data protection mechanisms
- improved monitoring of user activities

- **AI-Assisted Learning Tools**

Future versions of the system could integrate AI-based learning assistants that help students with problem solving, study planning, and learning recommendations.

5.2 Conclusion

The Home Tutor Finder System successfully demonstrates how modern web technologies can be used to create a centralized tutoring platform that connects students and tutors.

The system addresses common challenges faced by students when searching for qualified tutors and managing tutoring sessions. By integrating features such as tutor discovery, session scheduling, messaging, resource sharing, and virtual classrooms, the platform provides a comprehensive environment for online tutoring.

Although some limitations exist, the system provides a strong foundation for further development and improvement. With future enhancements such as AI-based tutor recommendations, mobile applications, and advanced notification systems, the platform has the potential to evolve into a fully scalable and intelligent tutoring solution.

The project highlights the importance of technology in improving access to education and demonstrates how digital platforms can support collaborative learning experiences.